



FACULTAD DE INGENIERÍA
UNIVERSIDAD NACIONAL DE LA PLATA

Informe final

FlowerpIoT:

Monitoreo de Ambiente de Plantas IoT

Taller de Proyecto 1
Ingeniería en Computación
Facultad de Ingeniería, Universidad Nacional de
La Plata

Mc Nally, Tomás Agustín
Ratusny, Jerónimo
Martini Morelli, Azul Angeles
Rodriguez Mesa, Mariano Agustin

15 de febrero de 2024

Índice

1. Introducción.....	3
2. Objetivos.....	4
2.1 Objetivos primarios.....	4
2.1.1 Diseño y ubicación de los componentes en la maceta.....	5
2.1.2 Calibración de sensores e implementación del código de lectura de los mismos....	5
2.1.3 Implementación de la comunicación mediante radio LoRa y protocolo WiFi.....	6
2.1.4 Almacenamiento y visualización de la información recopilada.....	6
2.2 Objetivos secundarios.....	6
3. Requerimientos.....	7
3.1 Funcionales.....	7
3.1.1 - Recolección de datos en campo.....	7
3.1.2 -Transmisión de Datos.....	7
3.1.3 - Visualización de datos.....	7
3.2 No Funcionales.....	7
3.2.1 - Recopilación de Datos.....	7
3.2.2 - Escalabilidad.....	7
3.2.3 - Disponibilidad.....	8
3.2.5 - Presupuesto.....	8
3.2.6 - Tiempo.....	8
3.2.7 - Hardware.....	8
3.3 De prueba.....	8
4. Diseño de hardware.....	8
4.1 Printed Circuit Board.....	8
4.1.1 Consideraciones generales.....	9
4.1.2 Tiras de pines (Pin headers).....	10
4.1.3 Diseño de componentes.....	11
4.1.3.1 Diseño del footprint para BH1750.....	11
4.1.3.2 Diseño del footprint para el módulo LoRa.....	11
4.1.3.3 Diseño del footprint para el módulo Relay.....	12
4.1.4 Diseño 3D.....	12
4.1.5 Diseño final.....	14
5. Diseño del software.....	16
5.1 Arquitectura y diseño del Firmware.....	16
5.2 Transmisión de datos.....	19
5.2.1 Máquina de estado de transmisión de datos.....	20
5.2.2 Gateway LoRa.....	21
6. Ensayos y mediciones.....	22
7. Conclusiones.....	24
7.1 Cumplimiento de requerimientos.....	26
7.1.1 Funcionales.....	26
7.1.1.1 Recolección de datos en campo.....	26
7.1.1.2 Transmisión de Datos.....	26
7.1.1.3 Visualización de datos.....	26

7.1.2 No Funcionales.....	27
7.1.2.1 Recopilación de Datos.....	27
7.1.2.2 Escalabilidad.....	27
7.1.2.3 Disponibilidad.....	27
7.1.2.4 Presupuesto.....	28
7.1.2.5 Tiempo.....	28
7.1.2.6 Hardware.....	28
7.1.3 De prueba.....	28
7.2 División de tareas final.....	29
7.3 Conclusión general.....	30
8. Bibliografía.....	32
9. Anexos.....	34
9.1 División de tareas.....	34
9.2 Cronograma final.....	37
9.3 Manual de Usuario.....	39
9.3.1 Carga de código en la EDU-CIAA-NXP.....	39
9.3.2 Conexionado y alimentación.....	41
9.3.3 Configuración y alimentación del Gateway.....	42
9.3.4 Configuración de TTS.....	42
9.3.5 Configuración del Servidor Node-RED.....	43
9.3.6 Configuración de la Base de Datos Firebase.....	44

1. Introducción

Las plantas ornamentales¹, a lo largo de los años, han sido apreciadas no solo por su valor estético, sino también por los beneficios ambientales y psicológicos que aportan a los espacios urbanos y hogares. La evolución tecnológica de nuestros tiempos ha transformado, de manera significativa, la forma en la que se aborda el cultivo, cuidado y comercialización de estas plantas.

En particular, para lograr un buen cuidado de la planta, se requiere tener información acerca de la misma, cosa que puede parecer sencilla en primera instancia debido a que se conoce el ambiente del vivero donde madura, pero esta información puede variar según la situación individual de cada planta, como por ejemplo dependiendo de su ubicación dentro del vivero o el estadio de crecimiento en el que se encuentra.

Es por esto que se plantea el desarrollo de un nodo inteligente, capaz de recolectar información relevante acerca de la planta en intervalos regulares de tiempo para poder generar, a partir del monitoreo del crecimiento de varias especies, una base de datos que sirva para establecer factores comunes en el desarrollo de enfermedades dentro de la planta, determinar la efectividad de una medida de control en comparación con otras empleadas, o, en el caso que sea posible, establecer las condiciones óptimas de desarrollo, entre otras cosas.

Se sabe que existen modelos en el mercado capaces de cumplir objetivos similares e incluso viveros completos automatizados que buscan los mismos resultados pero sus precios son demasiado elevados como para que puedan ser utilizados para el desarrollo local o doméstico .

La motivación de este proyecto radica en diseñar una maceta inteligente como la descrita que ofrezca una solución económica, fácilmente accesible y eficiente, en contraste con las alternativas actuales que tienen precios prohibitivos y escasa disponibilidad en el mercado.

En el último tiempo, se ha observado un rápido crecimiento del número de dispositivos de IoT (Internet de las Cosas) y sensores inalámbricos que transmiten datos a servidores a través de la nube. En estas plataformas de Cloud-Computing, la información es procesada y almacenada, permitiendo generar estadísticas en función del tiempo transcurrido.

El proyecto a realizar utiliza los conceptos de IoT además de agregar la ventaja de la comunicación entre nodos y el servidor a través del protocolo LoRa (Long Range), lo que permite la comunicación en redes de área amplia (WAN) con bajo costo de dispositivos, mantenimiento y consumo de energía aunque añade la limitación de la baja velocidad de datos, esto lo hace ideal para funcionar en entornos rurales, lugares donde suelen encontrarse los viveros. La comunicación requerida para poder conectar los diferentes componentes es realizada de forma unidireccional ya que, en primer lugar, el nodo inteligente realiza un procesamiento de los datos recibidos por los sensores

¹ Se conoce como planta ornamental a aquella que se cultiva y se comercializa con propósitos decorativos por sus características estéticas.

haciendo uso de la EDU-CIAA, luego se comunica utilizando una radio LoRa con el Gateway y por último la información se transmite a un Servidor en la nube a través de Wi-Fi.

2. Objetivos

2.1 Objetivos primarios

El objetivo principal de este proyecto es el diseño e implementación de un nodo inteligente que forme parte de un sistema de monitoreo de cultivos con tecnologías de IoT para recopilar datos cruciales que afectan el crecimiento de plantas.

El prototipo cuenta con 3 sensores los cuales estarán distribuidos en la maceta para lograr una medición lo más precisa posible, buscando obtener un conjunto integral de datos correspondientes a la situación actual de la planta. El sistema tiene como finalidad recolectar los datos de cada sensor utilizado, procesarlos y transmitirlos mediante el protocolo LoRa a un gateway para su posterior distribución a un servidor en la nube a través de WiFi, logrando así un modelo de bajo consumo, baja latencia y bajo mantenimiento además de ser escalable a mayor cantidad de sensores y nodos dentro del mismo sistema.

Para mayor organización, se decidió dividir el proyecto en 4 módulos:

- Diseño y ubicación de los componentes en la maceta.
- Calibración de sensores e implementación del código de lectura de los mismos.
- Implementación de la comunicación mediante radio LoRa y protocolo Wi-Fi.
- Almacenamiento y visualización de la información recopilada.

A continuación se presenta el diagrama en bloques del sistema a desarrollar:

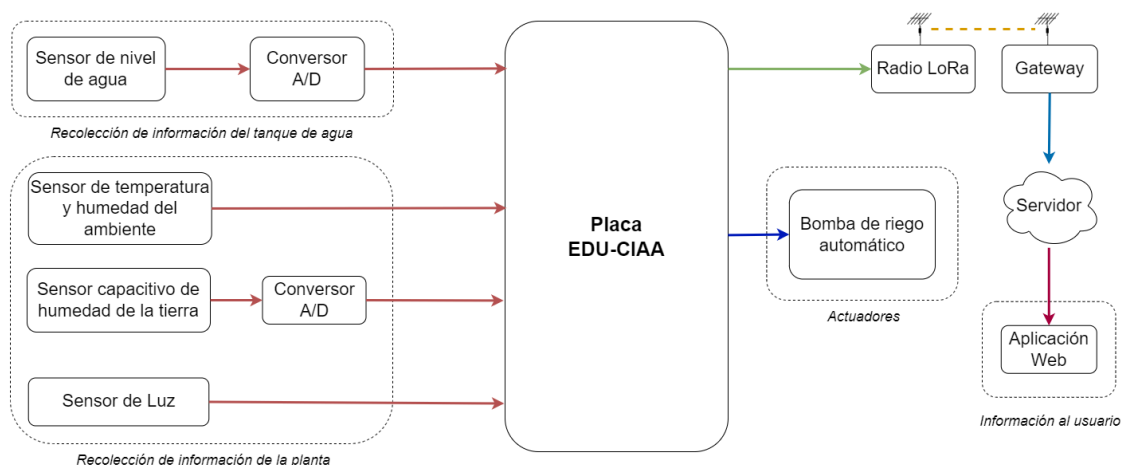


Figura 2.1: Diagrama en bloques del sistema propuesto²

2.1.1 Diseño y ubicación de los componentes en la maceta

El prototipo está orientado a plantas ornamentales de tamaño reducido, específicamente plantines, por lo que se busca realizar una maceta de no más de 30 cm de alto, 30cm de ancho y 30cm de profundidad, dejando suficiente lugar para el desarrollo de la planta e instalación de los diferentes sensores en la maceta.

Además se prevé un lugar adicional de 10 cm de alto, 30 cm de ancho y 30 cm de profundidad donde se almacenará la placa EDU-CIAA-NXP basada en LPC4337, de 137 x 86 mm y los circuitos necesarios para acondicionamiento de las señales recibidas por los sensores.

2.1.2 Calibración de sensores e implementación del código de lectura de los mismos

El nodo inteligente dispondrá de 3 sensores en total, los cuales recopilan información acerca de:

- Humedad y temperatura del ambiente
- Humedad de la tierra
- Niveles de luz.

Cada uno de estos sensores debe ser calibrado de forma precisa para conocer el rango de medida y poder desechar las mediciones que no se encuentren en este rango además de establecer valores normales que toma la planta en un determinado momento del día y bajo determinadas características del clima.

Para realizar esta calibración se tomarán mediciones en condiciones de reposo y en condiciones “extremas”, es decir, condiciones límites en las que se pueda llegar a encontrar la planta.

Además, se implementará un algoritmo de lectura capaz de recolectar los datos cada cierto intervalo de tiempo, para transmitirlo de manera conjunta junto con la hora

² Tanto en el diagrama en bloques como en el diagrama esquemático se agregan sensores y componentes que no se encuentran contemplados en los objetivos primarios pero si en los objetivos secundarios para, en caso de poder alcanzarlos contar con el proceso documentado.

en la que se tomaron. Principalmente se hará hincapié en el uso de sensores con modos de comunicación diferentes con el objetivo de mostrar las diferentes maneras en la que se pueden relevar las variables relativas a la planta.

2.1.3 Implementación de la comunicación mediante radio LoRa y protocolo WiFi

La implementación de la comunicación mediante radio LoRa es un componente crucial en nuestro sistema de monitoreo. Después de recopilar la información de todos los sensores, es fundamental establecer una forma eficiente y confiable de transmitir estos datos hacia un servidor centralizado para su posterior procesamiento y análisis. Para lograrlo, se diseñará un sistema de comunicación que integra tanto la tecnología LoRa como la conexión Wi-Fi.

El gateway LoRa, actuando como intermediario y concentrador, recibe los datos del nodo y los envía a través de una conexión Wi-Fi hacia un servidor en la nube.

2.1.4 Almacenamiento y visualización de la información recopilada

Una vez establecida la conexión entre el gateway LoRa y el servidor, la información recopilada será almacenada en una base de datos de forma ordenada. La implementación de este sistema de almacenamiento facilitará el procesamiento y análisis de dicha información.

El resultado de este análisis se podrá visualizar de forma clara mediante una aplicación web de fácil acceso para el usuario.

2.2 Objetivos secundarios

Una vez finalizados los objetivos esenciales del proyecto, se tiene previsto para un futuro agregar posibles mejoras. En primer lugar, se plantea la posibilidad de implementar acciones de control basadas en los datos recopilados, como por ejemplo el desarrollo de un sistema de riego automático que se active al detectar que la humedad del suelo no es adecuada para el crecimiento de la planta agregando nuevos sensores que permitan monitorear el manejo de la bomba de agua encargada del riego, o una luz natural que se active cuando detecte que el nivel de luz recibido por la planta no es el suficiente.

En conjunto con las acciones de control se prevé agregar un seleccionador de modo que permita a la maceta trabajar en modo de solo monitoreo o también realizando las acciones de control previamente mencionadas. Además, para obtener una información más completa de la planta, en los casos donde sea necesario, es posible el agregado de nuevos sensores, como por ejemplo de pH o salinidad.

Adicionalmente, se propone una aplicación para dispositivos móviles que permita una visualización de los datos recolectados, ofreciendo así una experiencia más amigable con el usuario.

Por último, una visualización en un display permitiría conocer el estado de la planta de forma integral sin requerir de la aplicación, logrando aumentar su utilidad en ambientes sin conexión.

3. Requerimientos

3.1 Funcionales

3.1.1 - Recolección de datos en campo

- Recibir el valor de la humedad del suelo medido por el sensor
- Realizar el sensado del valor de la humedad del suelo de forma periódica.
- Recibir el valor de la humedad del ambiente medido por el sensor
- Realizar el sensado del valor de la humedad del ambiente de forma periódica.
- Recibir el valor de la temperatura medido por el sensor
- Realizar el sensado del valor de la temperatura de forma periódica.
- Recibir el valor del nivel de luz medido por el sensor
- Realizar el sensado del valor del nivel de luz de forma periódica..

3.1.2 -Transmisión de Datos

- Establecer una comunicación con el gateway.
- Establecer una comunicación entre el gateway y el servidor.
- Enviar datos recopilados al servidor en tiempo real.

3.1.3 - Visualización de datos

- Lograr recibir a través del gateway información de niveles de luminosidad.
- Lograr recibir a través del gateway información de niveles de humedad del suelo.
- Lograr recibir a través del gateway información de niveles de luz detectados.
- Una vez almacenados los valores, mostrar en la aplicación web, un dashboard con información actual de los sensores

3.2 No Funcionales

3.2.1 - Recopilación de Datos

- Garantizar una baja latencia en la recopilación y transmisión de datos.
- Minimizar el consumo de energía para prolongar la duración de la batería.

3.2.2 - Escalabilidad

- Permitir la expansión del sistema para monitorear múltiples campos de cultivo simultáneamente.
- Garantizar que el sistema pueda manejar un crecimiento significativo en el volumen de datos.

3.2.3 - Disponibilidad

- Asegurar una alta disponibilidad del sistema para evitar interrupciones en el monitoreo.
- Establecer un plan de recuperación ante errores para mitigar fallos críticos.
- Garantizar que los sensores sean reemplazables y calibrables de forma sencilla.

3.2.5 - Presupuesto

- Mantener el desarrollo dentro del presupuesto limitado disponible

3.2.6 - Tiempo

- Alcanzar un producto finalizado en el tiempo limitado que se tiene

3.2.7 - Hardware

- Utilizar la placa educativa EDU-CIAA-NXP para el desarrollo del prototipo

3.3 De prueba

- Documentar el proceso de calibración de sensores y configuración del sistema.
- Verificar que la comunicación Wi-Fi sea estable y no se pierda la conexión durante un periodo prolongado.
- Evaluar que el nodo inteligente pueda comunicarse con el gateway LoRa.
- Asegurarse que el servidor en la nube pueda recibir, almacenar y gestionar los datos recibidos.
- Verificar que los datos almacenados se muestran correctamente en la plataforma web.
- Comprobar que los datos se actualicen en tiempo real.

4. Diseño de hardware

En base al diagrama en bloques de la Figura 2.1 y a los requerimientos mencionados se estudiaron diferentes tipos de sensores con el objetivo de utilizar los que mejor se adaptan al prototipo de solución a desarrollar, las conclusiones obtenidas y decisiones tomadas se encuentran bajo la sección Anexo 1: Sensores a utilizar. Una vez que los sensores fueron definidos, se procedió a elaborar el esquemático que conecta dichos sensores a la EDU-CIAA, junto con la placa de circuito impreso que materializa este esquemático físicamente. A continuación se detalla el resultado final del esquemático y los pasos seguidos para la elaboración física del PCB (Printed Circuit Board).

4.1 Printed Circuit Board

El circuito esquemático desarrollado no sufrió cambios en relación al Informe de Avance 2 y su interconexión con los sensores está explicada en el Anexo 1: Sensores a utilizar, a su vez una imagen final del esquemático se encuentra presente en el Anexo 2.1: Diagrama esquemático. Es por esto que en la presente sección solo se detallará las consideraciones para la realización del PCB.

4.1.1 Consideraciones generales

A diferencia del circuito esquemático, la PCB si sufrió modificaciones de diseño relacionadas a correcciones realizadas, es por esto que a continuación se detallarán las características de modelado del diseño final.

En todos los componentes se reemplazaron los pads definidos previamente por otros de forma cuadrada y/o rectangular sugeridos por la cátedra, facilitando así el posterior proceso de perforación para el caso de pads para pines macho y soldadura en general.

Los pads donde se ubicaran posteriormente pines macho tienen dimensiones de 40th (1.016 mm) para el diámetro interno y de 80th por 80th (2.032 mm x 2.032 mm). En cambio, para los pads de forma rectangular, los mismos poseen un diámetro interno de 30th (0.762 mm) con un tamaño de 70th por 0.1in (1.778 mm por 2.54 mm).

Las pistas utilizadas tienen un ancho de T40 (1.016 mm) y las vías poseen nuevas dimensiones de 30th como diámetro interno (0.762 mm) y 0.12in de diámetro externo (3.048 mm).

Las borneras, por otro lado, utilizan pads cuadrados de 0.15in por 0.15in (3.81 mm por 3.81 mm) con un diámetro interno de 65th (1.651 mm).

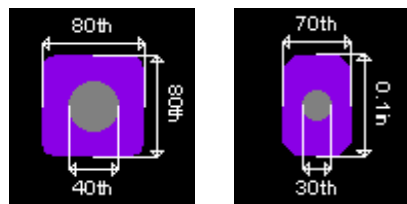


Figura 4.1.1a: Configuración utilizada para pads

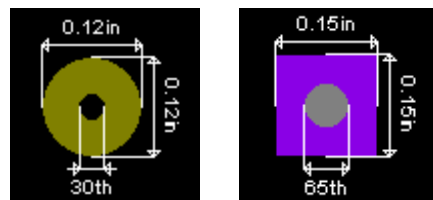


Figura 4.1.1b: Configuración utilizada para vías (izquierda) y pads de las borneras (derecha)

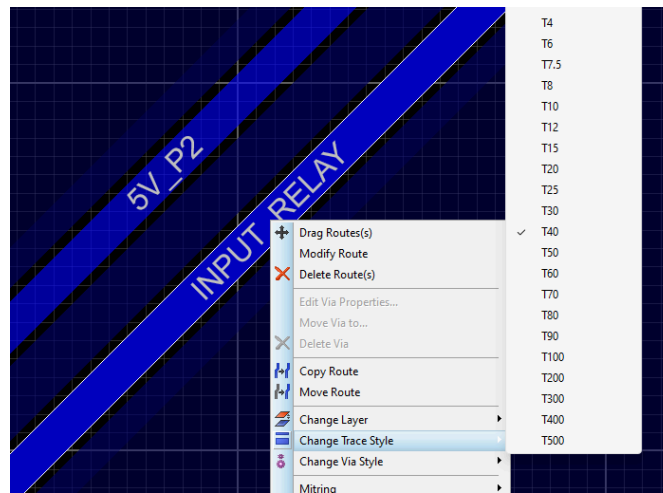


Figura 4.1.1c: Configuración utilizada para pistas

Se consideraron los requerimientos mínimos de separación entre las pistas de 0.7 mm.

Fue requerida una única resistencia TH (thru hole) para conectar en paralelo con el sensor DHT11 de 4.7 kohm. Su footprint asociado se muestra a continuación con un tamaño de 0.4in (10.16 mm) y pads del tamaño antes descrito (70th por 0.1in), esto es debido a que al tener que soportar una corriente máxima de 702 μA y por lo tanto 2.3mW, se necesita una resistencia de 1/8W, entonces, con el objetivo de lograr intercambiabilidad en caso que se requiera conectar a 5V, se decidió utilizar una de 1/4 W.

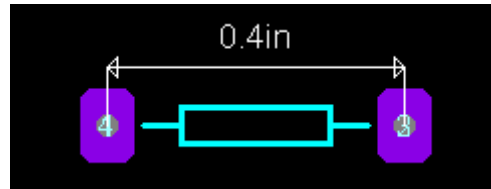


Figura 4.1.1d: Footprint asociado a la resistencia de 4.7 kohm

4.1.2 Tiras de pines (Pin headers)

Para el esquema de la tira de pines de J1 y J2 se utilizó como base el componente 10073456-049LF de Proteus. Teniendo en cuenta los sensores y/o componentes en general del proyecto, se eliminaron los pines no utilizados.



Figura 4.1.2a: Tira de pines J1, correspondiente a P1 en EDU-CIAA

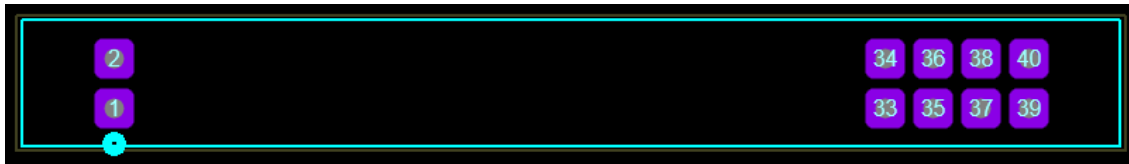


Figura 4.1.2b: Tira de pines J2, correspondiente a P2 en EDU-CIAA

También se dejaron pads como lo son el 3, 4, 9, 39, 40 del J1 y los 34, 35, 37, 38, 40 del J2 para garantizar una sujeción estable del poncho.

4.1.3 Diseño de componentes

Como la biblioteca de componentes de Proteus no cuenta con todos los diseños que fueron requeridos para el desarrollo del PCB (footprints) algunos de ellos fueron diseñados e implementados, en las siguientes secciones se mencionan los que se diseñaron y de que forma se realizó.

4.1.3.1 Diseño del footprint para BH1750

Para el sensor de luz que cuenta con 5 pines (GND, ADD, I2C_SDA, I2C_SCL, VCC) con una distancia entre ellos de 0.1in (25,4 mm) se diseñó el siguiente footprint teniendo en cuenta las dimensiones del mismo.

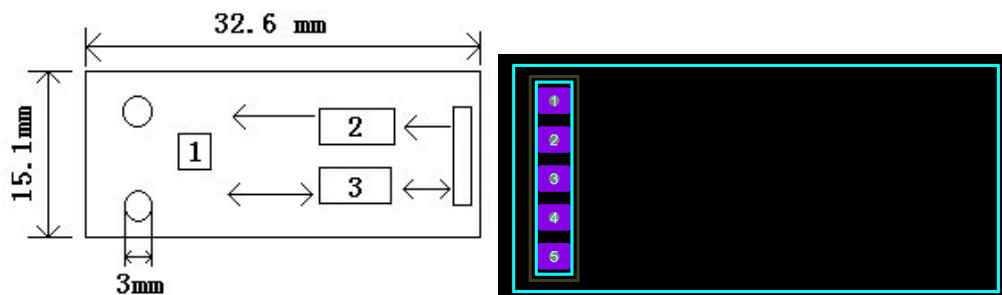


Figura 4.1.3.1a: Footprint y dimensiones del sensor BH1750

4.1.3.2 Diseño del footprint para el módulo LoRa

El footprint del módulo LoRa que se utilizó fue pensado teniendo en cuenta todos los pines que este posee aunque no sean utilizados en su totalidad, para así garantizar una firme sujeción del mismo. Fueron cambiados los números de los pines utilizados (1, 2, 3 y 4) para que sea coherente con el esquemático y el módulo LoRa. La separación entre cada pin es la misma que para el sensor BH1750, es decir, de 0.1in (25,4 mm) entre cada uno.

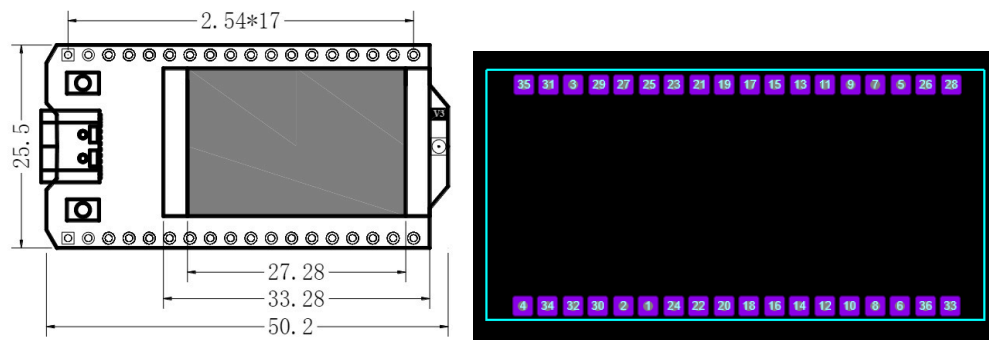


Figura 4.1.3.2a: Footprint y dimensiones del módulo LoRa

4.1.3.3 Diseño del footprint para el módulo Relay

En particular para el módulo relay, se necesitaron 3 pines, VCC, GND, INPUT_Relay. La utilización de un módulo que ya tiene incluido el relay, los leds, las resistencias, la bornera y transistor requeridos, simplificó en gran medida el diseño del footprint. La separación entre cada pin es la misma utilizada para los anteriores diseños, 0.1in (25,4 mm) entre cada uno.

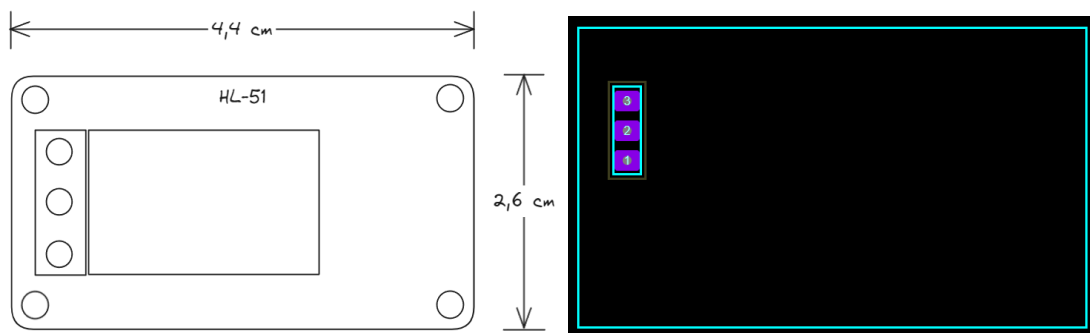


Figura 4.1.3.3a: Footprint y dimensiones del módulo Relay

4.1.4 Diseño 3D

Para finalizar con el desarrollo del PCB se realizó el diseño 3D de la PCB, este diseño incluyó el agregado de diferentes modelos 3D como el de los sensores BH1750 y DHT11 así como también la inclusión del módulo LoRa, el módulo Relay y dos tiras de tres pines hembra, cada uno de estos modelos fue agregado al proyecto y luego ajustado para coincidir con el lugar exacto donde se ubicará el componente. Los archivos .stp de los modelos 3D presentes en el PCB se pueden encontrar en la siguiente carpeta:

Modelos 3D

Para poder visualizar el modelo de forma completa con todos los modelos 3D agregados, primero se deben descargar los mismos y colocarlos en la carpeta MCAD (Archivos de programa, Labcenter Electronics, Proteus 8 Professional, DATA, MCAD).

Una vez hecho esto, debemos enlazarlos con los modelos correspondientes en Proteus, seleccionando el modelo y luego “3D Models”.

Una visualización más rápida de las vistas delantera e izquierda se pueden observar en las Figuras 4.1.4a y 4.1.4b. Además, más imágenes tanto del PCB como el diseño 3D

junto con los archivos pdf de las vistas y los archivos necesarios para replicar el modelo se encuentran en el Anexo 3.

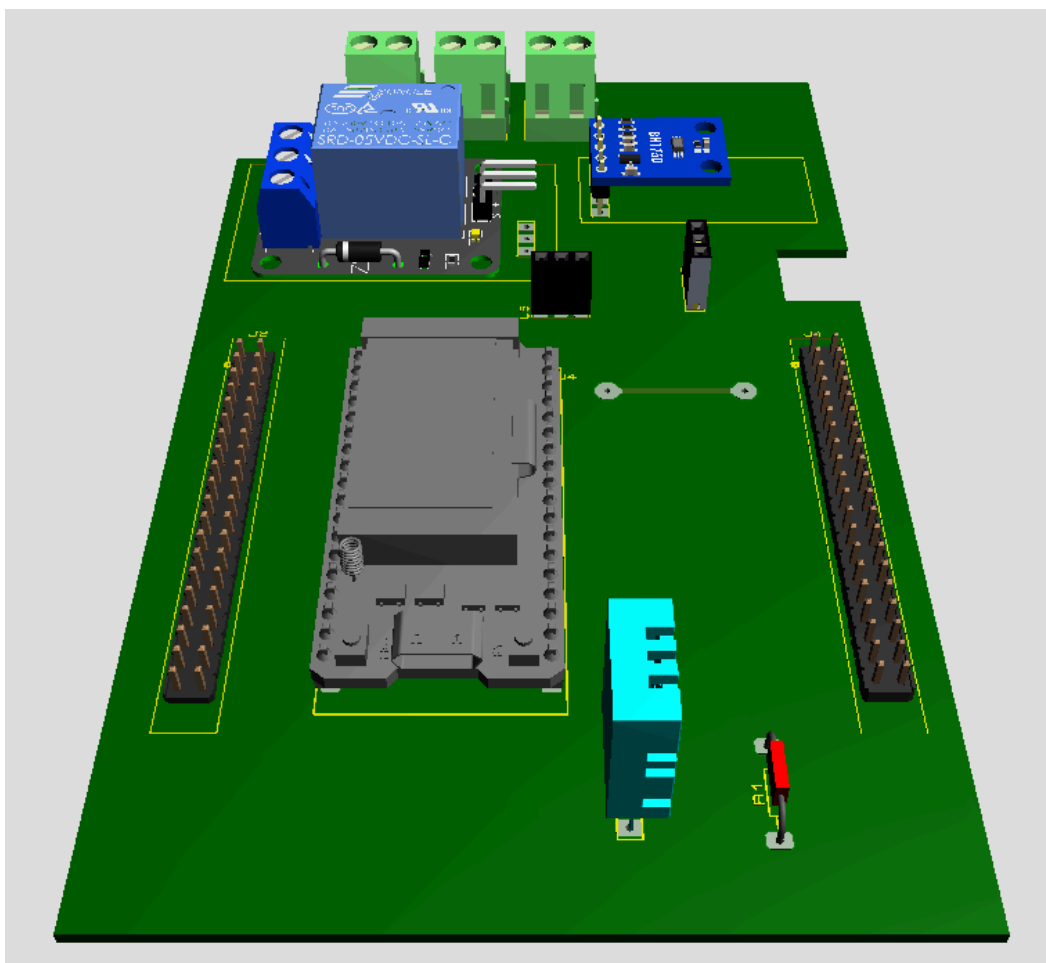


Figura 4.1.4a: Vista frontal del modelo 3D

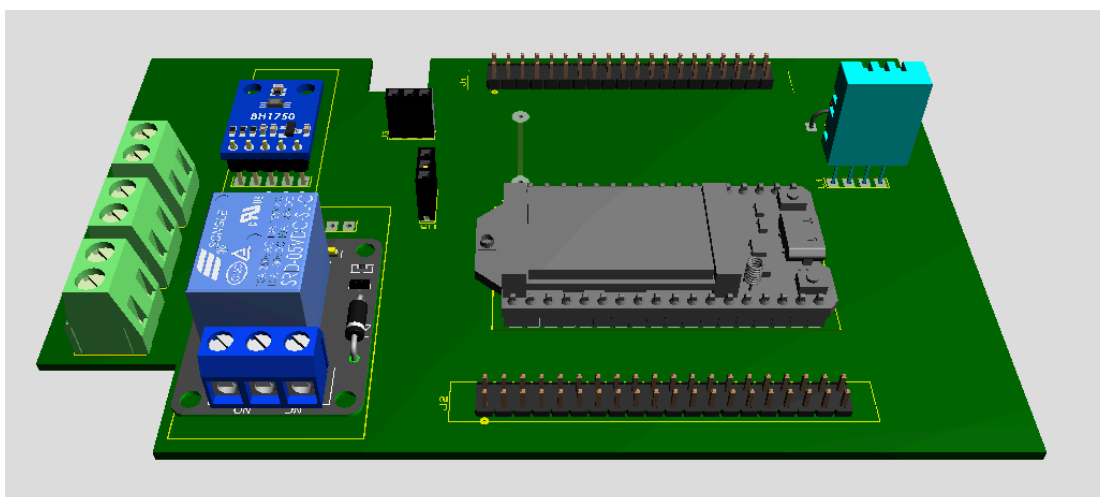


Figura 4.1.4b: Vista lateral izquierda del modelo 3D

4.1.5 Diseño final

En el diseño final del PCB se pueden observar un total de dos cortes realizados para permitir el acceso a la placa utilizada. El corte ubicado en el lado superior izquierdo (viendo la figura 4.1.5a) fue utilizado para garantizar la alimentación. Y el segundo corte, ubicado en el lado derecho debajo del sensor BH1750, fue realizado debido a que allí se encuentra el botón de “Reset” de la misma.

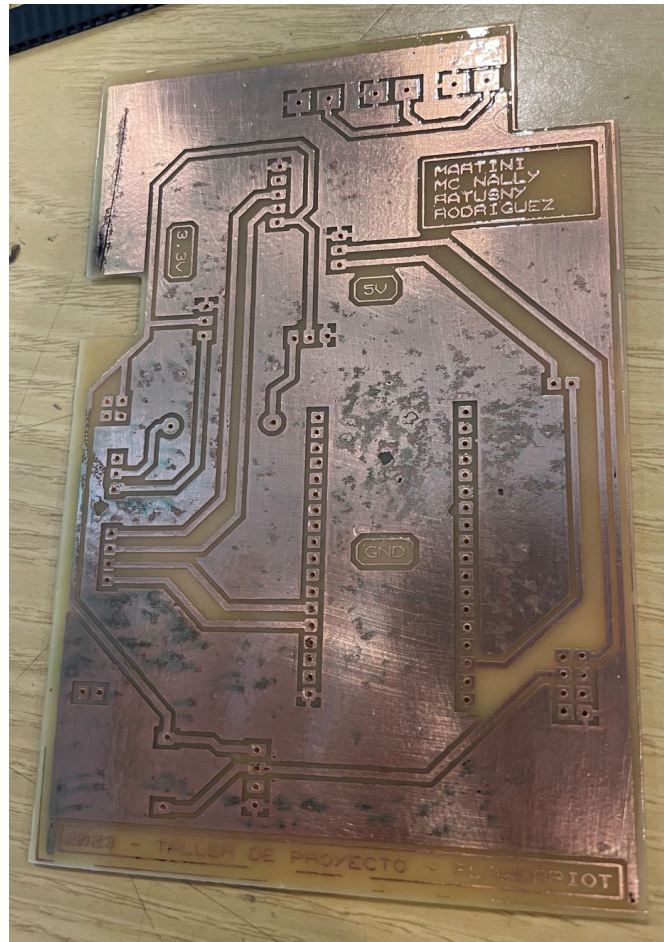


Figura 4.1.5a: Diseño final en placa de cobre simple faz

Se agregaron detalles al PCB tales como, los nombres de los integrantes del equipo, el nombre de la materia, el año y el nombre del proyecto.

Luego de imprimir el diseño y traspasarlo a la placa, se perforó la misma y se “pintó” con flux para protegerla. Posteriormente se realizó la soldadura de los componentes solicitados, esto puede apreciarse en las Figuras 4.1.5b y 4.1.5c las cuales muestran las vistas superior e inferior de la placa con los componentes ya soldados

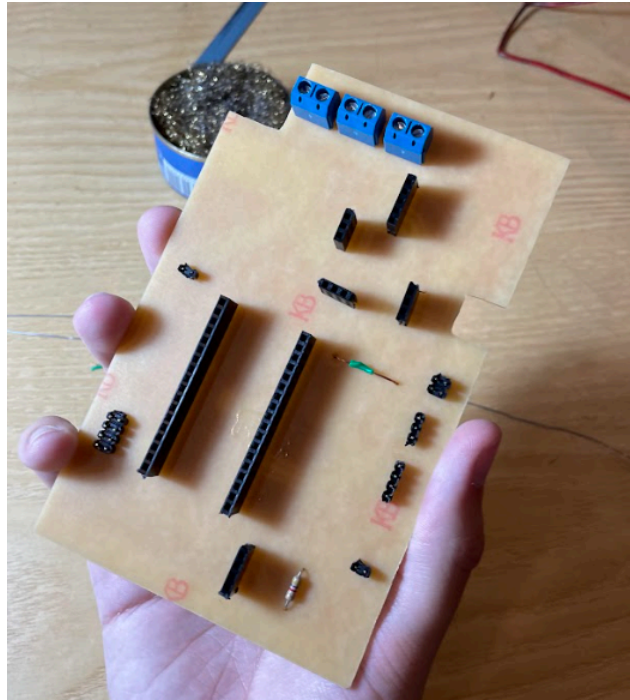


Figura 4.1.5b: Vista superior del PCB final con headers, borneras y resistencias soldadas

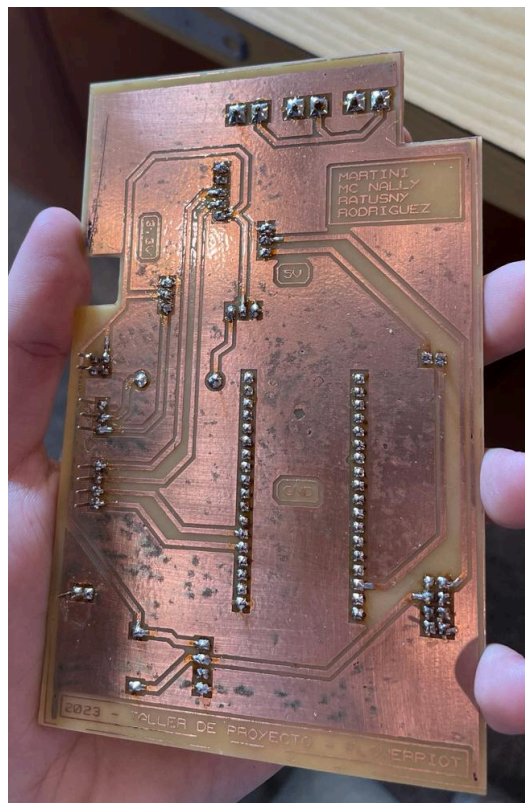


Figura 4.1.5c: Vista inferior del PCB final con headers, borneras y resistencias soldadas

5. Diseño del software

Para explicar el diseño general del software, se divide la explicación en dos tópicos bien diferenciados. Por un lado, la arquitectura y diseño del firmware donde se detalla el enfoque que se llevó a cabo para realizar la temporización para la planificación de eventos (en este caso la recolección y envío de datos) y por otro lado, el proceso tras obtener los datos y haberse enviado por radio LoRa. La topología de red utilizada es una topología de tipo estrella, para más información acerca de la misma puede consultarse el Anexo 2: Información relativa al diseño del software.

5.1 Arquitectura y diseño del Firmware

La arquitectura del firmware propuesta tendrá un enfoque del tipo Time - Triggered, la cual se basa en una planificación de tareas disparadas por tiempo mediante una única interrupción periódica de Timer (comúnmente llamada RTI)³.

Cuando no se deba ejecutar ninguna tarea, se pondrá al MCU en modo bajo consumo (SLEEP) hasta el próximo tick y de esta forma se ahorrará energía. Este tipo de arquitectura está compuesta principalmente por un planificador de tareas (Scheduler) que permite decidir qué tarea corresponde ejecutar en base a la temporización basada en una RTI, y un despachador de tareas (Dispatcher) que permite ejecutar las tareas planificadas con distintas prioridades.

A partir de esta configuración, se establece la base para lo que comúnmente se conoce como un RTOS (Real Time Operating System), al que se llamará sEOS (simple embedded operating system) de ahora en adelante.

Teniendo en cuenta los requisitos del proyecto, tras la evaluación de diversas alternativas de arquitecturas para el firmware del mismo, se llegó a la conclusión de que la arquitectura Time-Triggered es la más adecuada, dado que permite planificar y ejecutar la recopilación de datos en intervalos regulares de tiempo, garantizando una gestión eficiente de los recursos y la oportunidad de transmitir los mismos una vez que son recolectados. Cabe destacar que este enfoque es comúnmente utilizado en sistemas automatizados y en general, en contextos donde es necesario programar y ejecutar tareas de manera planificada.

Las arquitecturas de tipo Time-Triggered están diseñadas en torno a un bucle principal en el que se gestiona la ejecución de tareas planificadas a través del dispatcher, como puede observarse en la Figura 5.1a.

³ Al igual que el caso de la Topología de Red, más información del enfoque del firmware puede encontrarse en el Anexo 2.

```

// Inicializo el planificador
schedulerInit()

// Agrego una tarea
schedulerAddTask(CapturaYEnvioDatos, tiempo_en_ticks)

// Inicializo el planificador con un intervalo X ms cada interrupción
schedulerStart(X)

Bucle infinito

    // Llamo al despachador de tareas
    schedulerDispatchTasks()

fin bucle infinito

```

Figura 5.1a: Pseudocódigo principal del firmware propuesto

La misma se destaca por su enfoque sencillo, ya que se centra en la ejecución de una única tarea a despachar. Gracias a esto, se reduce la complejidad del firmware, lo que facilita el mantenimiento y realización del mismo.

```

// Dispatcher
schedulerDispatchTasks() {
    Si se activó el flag {
        // Tarea que recolecta y envía los datos leídos por los sensores
        CapturaYEnvioDatos()
        // Borrar valor del flag leído
        Flag_Sensores = 0
    }
}

```

Figura 5.1b: Pseudocódigo del despachador de tareas

La función *schedulerDispatchTasks()* es la responsable de gestionar la ejecución de dicha tarea, verificando si el flag (*Flag_Sensores*) ha sido activado, luego de completar la tarea de *CapturaYEnvioDatos()* el mismo se reinicia estableciendo su valor en cero nuevamente.

```

// Scheduler

schedulerUpdate() {

    Si se llegó al valor determinado de tiempo

    // Se activa el flag de la tarea

    Flag_Sensores = 1

    // Se reinicia el conteo

}

}

```

Figura 5.1c: Pseudocódigo del planificador de tareas

El planificador emplea las funciones disponibles en la biblioteca sAPI para gestionar el tiempo de ejecución de la tarea. Este control de tiempo se lleva a cabo mediante el conteo de ticks del sistema definidos en un tiempo determinado (1 ms), lo que posibilita la ejecución simultánea de otras tareas. En lugar de quedarse inactivo esperando a que se complete el tiempo, las funciones de la biblioteca *seos_pont_2014_scheduler* simplemente verifica si el tiempo predeterminado ha transcurrido, lo que permite al sistema continuar realizando otras operaciones.

```

// Función que obtiene y envía los datos

CapturaYEnvioDatos() {

    // Recolección de datos de los sensores

    Datos = LeerSensores()

    // Envío de datos al nodo LoRa (mediante UART)

    EnviarDatos(Datos)

}

```

Figura 5.1d: Pseudocódigo de la tarea

Finalmente, se muestra en la figura 5.1d el pseudocódigo que describe la recolección y envío de datos desde los sensores conectados a la EDU-CIAA hacia el nodo LoRa.

Las funciones de la biblioteca *seos_pont_2014_scheduler* se encuentran explicadas en el Anexo 2.

Una vez descrita la arquitectura del firmware a seguir se procede con el diseño del firmware en sí mismo. Este se encuentra basado en la biblioteca sAPI, que proporciona una capa de abstracción respecto del hardware como la que se puede observar en la Figura 5.1e.

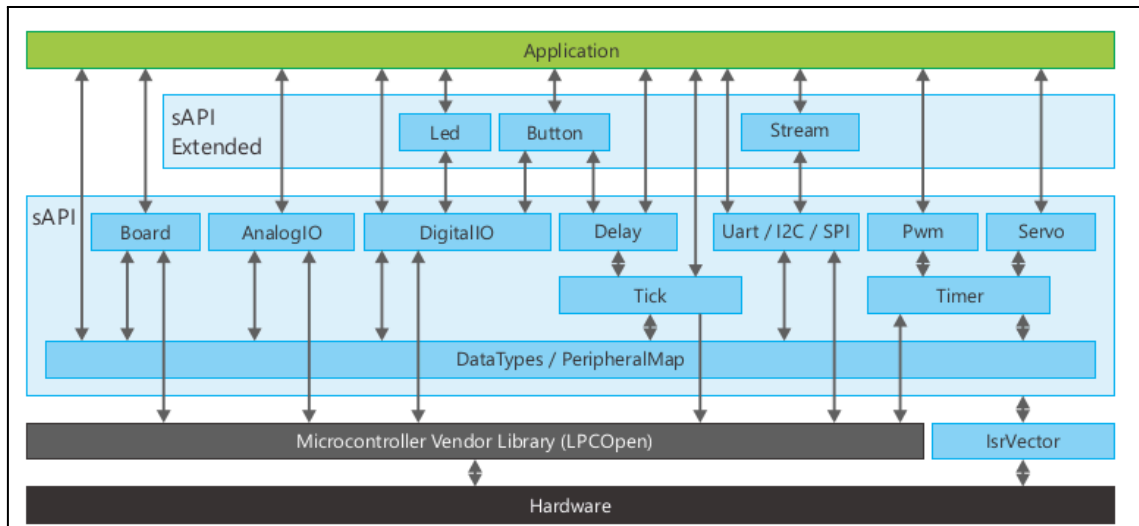


Figura 5.1e: Capas del firmware proporcionado por sAPI

Para cada uno de los componentes utilizados a lo largo del apartado 4, existe un módulo controlador como se puede observar en el diagrama de la Figura 5.1f.

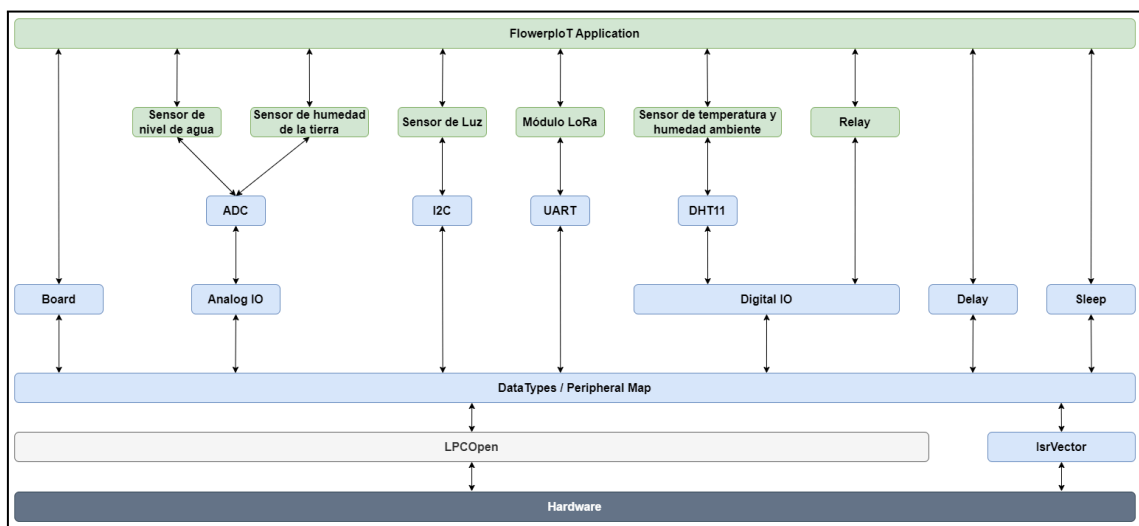


Figura 5.1f: Capas del firmware de FlowerIoT (en verde) basado en sAPI (en azul)

Estas bibliotecas necesarias para el desarrollo del firmware se encuentran en el Anexo 2 junto con una explicación acerca del objetivo dentro del proyecto y las funciones que provee. La versión final del código desarrollado puede consultarse en:

[Codigo Final](#)

5.2 Transmisión de datos

El nodo LoRa recibe los datos capturados por los sensores conectados a la EDU-CIAA mediante la transmisión UART de los datos, con un formato de tipo string que simula ser de tipo JSON.

La placa de desarrollo que se utilizará dispone de un ESP32 el cual facilita su programación con bibliotecas ya existentes o que provee el fabricante de la placa, por lo que no se desarrolla directamente sobre la EDU-CIAA, sino que se realizará todo lo

relacionado a LoRaWAN en el ESP32 y ambas placas, estarán conectadas mediante los pines de transmisión y recepción en serie (Tx y Rx en la EDU-CIAA). Además se conecta GND del módulo WiFi Lora 32 al GND de la EDU-CIAA para que ambos tengan el mismo valor de referencia.

```
{
  "temperature": 23.45,
  "humidity": 65.78,
  "light": 1200.50,
  "water_level": 75,
  "soil_moisture": 50
}
```

Figura 5.2: Mensaje JSON enviado desde la EDU-CIAA a la ESP32

5.2.1 Máquina de estado de transmisión de datos

Para comprender mejor el funcionamiento de la transmisión de datos, tenemos el siguiente diagrama, donde se pueden ver los distintos estados por los que va pasando el módulo LoRa:

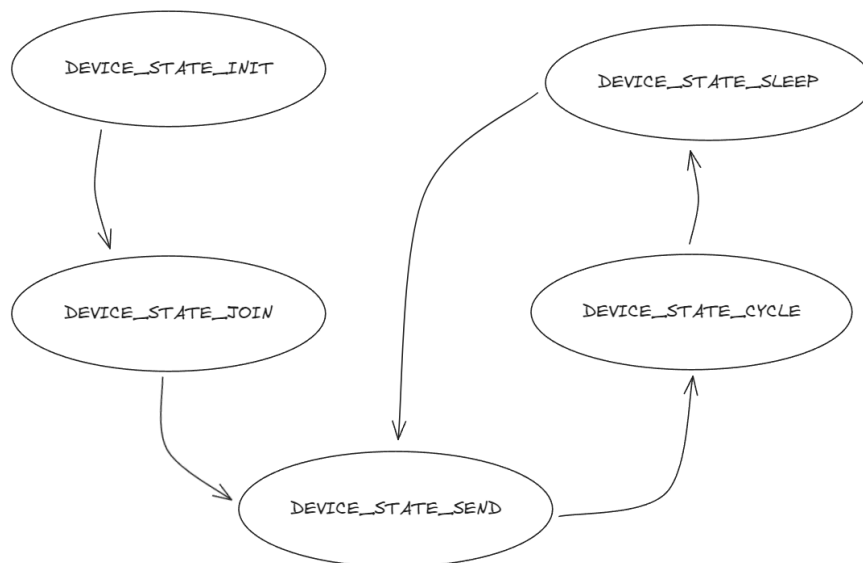


Figura 5.2.1: Diagrama de estados de la transmisión LoRa.

En primer lugar se inicializa el dispositivo en el estado “DEVICE_STATE_INIT”, para ello se configuran algunos parámetros como la utilización de OTAA o ABP, la clase previamente explicada, la región de operación, entre otras cosas. Luego, en el estado “DEVICE_STATE_JOIN”, realiza el intento de unirse a la sesión para la transmisión de datos, en caso de utilizar OTAA. Para nuestro caso, como hacemos uso de ABP, no requiere hacer unión y se lo saltea. Esta sección fue implementada igualmente para futuras mejoras.

Una de las partes más importantes es el envío en el estado “DEVICE_STATE_SEND”, el cual se realiza únicamente si el puerto serie contiene información, en caso de que contenga información, prepara el paquete mediante una

función, para posteriormente enviarla. En caso de que no haya nada para transmitir, pasa al estado “DEVICE_STATE_CYCLE”, que se encarga de calcular el proximo envío de mensaje.

Actualmente, las pruebas realizadas, fueron hechas teniendo en cuenta el estado “DEVICE_STATE_CYCLE” que calcula los envíos de paquete, es decir, transmite cada un tiempo determinado. El cálculo del tiempo proviene de una suma entre la variable “appTxDutyCicle” que es el tiempo que queremos que pase entre cada envío y un número aleatorio que está en un rango que provee la biblioteca. Una mejora es modificar este estado, para que transmita cuando reciba por UART los datos enviados por la EDU-CIAA.

Por último, el estado “DEVICE_STATE_SLEEP” pone en bajo consumo el dispositivo mientras no tenga para transmitir.

5.2.2 Gateway LoRa

El modelo disponible para la realización del proyecto es el ‘WisGate Edge Lite’. Este modelo en específico cuenta con 8 canales y con tres interfaces de conexión a Internet: vía Ethernet, vía Wi-Fi y conexión de datos móviles. Además, soporta *MQTT Bridge mode* y TLS para el cifrado de las comunicaciones, para más información ver las secciones de MQTT y TLS en el Anexo:LoRa. Este modelo es ideal para el proyecto debido a que se prevé tener múltiple cantidad de nodos, es decir, múltiples la cantidad de macetas con sensores. Si el gateway fuera Single-Channel, se necesitaría un gateway por nodo lo que haría muy costosa la escalabilidad del número de nodos.



Figura 5.2.2: Imagen del modelo WisGate Edge Lite.

Una vez introducido el gateway a utilizar, LoRa posee múltiples modos de funcionamiento, que serán explicados en el Anexo de LoRa. Por lo tanto es de mucha importancia el cómo configuramos este gateway para poder recibir los paquetes que envíe el nodo LoRa, como también la forma en la que mandamos esos paquetes a nuestro servidor para posteriormente almacenar los datos en una base de datos. Para más información acerca de las tecnologías y/o servicios utilizados, se encontraran en un apartado específico en el Anexo 1.7.

En primer lugar, una vez que el nodo LoRa envía los paquetes y los recibe el gateway, hacemos uso de un servicio/servidor de LoRaWAN llamado TTS (The Things Stack) que actúa como un servidor de calidad empresarial, gestionando millones de dispositivos LoRaWAN en producción. Al integrar TTS en el circuito de transmisión y

almacenamiento de datos, se alivia la carga del gateway, permitiendo un procesamiento más rápido de mensajes y nodos.

Luego, teniendo en cuenta que, habrá un servidor corriendo, se optó por la utilización de Node-RED. Este se encargará de recibir los paquetes MQTT enviados por TTS, procesarlos y mostrarlos en un dashboard. Utiliza un cliente MQTT para suscribirse a los temas relevantes, recibiendo automáticamente los mensajes de TTS. Los datos recibidos se decodifican, transformándolos en JSON para su procesamiento y visualización en medidores en tiempo real.

Para finalizar el circuito, Node-RED se encargará de enviar esos paquetes que recibe de TTS a una base de datos no relacional, como lo es Firebase, que es un almacenamiento limitado de google, que permite una alta escalabilidad en caso de tener múltiples nodos. Además, proporciona la estructura para almacenar los datos junto con marcas de tiempo, permitiendo su visualización y comparación posteriormente. Se utiliza el plugin "node-red-contrib-firebase" en Node-RED para obtener datos de Firebase. Los datos obtenidos se formatean en JSON antes de mostrarlos en gráficos, asegurando que la hora del gráfico corresponde a la marca de tiempo de los datos almacenados en Firebase.

6. Ensayos y mediciones

Una vez finalizado el desarrollo del software se verificó el funcionamiento de los sensores acorde a lo descrito según las hojas de datos. En el Anexo 3: “Ensayos sobre componentes sin PCB” es posible observar los diferentes códigos y resultados obtenidos al realizar pruebas sobre los componentes sin hacer uso de la PCB. Una vez que se construyó el PCB, habiendo probado cada uno de los componentes de forma individual se procedió al ensayo de todo el sistema en simultáneo. Una video que refleja esta prueba puede encontrarse en el siguiente enlace:

 con PCB

En el video es posible observar el funcionamiento en conjunto de todos los componentes. El programa de prueba cargado cumple la función de enviar cada dos segundos los valores detectados de las variables a través de LoRa y además, en el caso que detecta más de un cierto valor de agua a través del sensor HW-038 encender la bomba de agua por cuatro segundos. Cómo es posible observar se está haciendo uso de una protoboard aunque su única función es facilitar la interconexión de la bomba de agua al resto del sistema.

El tiempo que tarda la aplicación web de node-red en actualizarse a partir del envío de nuevos datos a través de LoRa ronda los treinta segundos lo que consideramos como un tiempo más que aceptable considerando el uso planteado para el sistema.

Por último, vale la pena aclarar que el código utilizado en la muestra no es el mismo que el desarrollado en el firmware ya que con el objetivo de poder observar cambios en tiempos más razonables y lograr una presentación más interactiva se modificaron las variables sensadas y el tiempo de acción de la bomba entre otras cosas

En cuanto a los problemas o dificultades encontrados se tuvieron varios relacionados al armado del PCB mientras que no se tuvieron demasiadas dificultades en lo referido al armado del firmware. Los problemas encontrados se detallan en la siguiente lista numerada.

1. En primer lugar, al momento de realizar el marcado del PCB para cortarla se produjo una raya involuntaria que supondría un corte en las pistas de cobre, este problema fue solucionado al orientar de forma diferente a la originalmente planteada la placa.
2. En segundo lugar hubo algunos problemas en cuanto a los componentes, uno de ellos el módulo relay el cual encendía la luz indicadora de funcionamiento pero al recibir la señal para “switchear” entre contactos no funcionaba correctamente, la solución a este problema fue utilizar un nuevo módulo relay pero trajo como consecuencia que los pines hembra no se encontraban en la posición correcta y por lo tanto el módulo relay tuvo que ser conectado mediante cables. De igual manera, el sensor DHT11 tenía problemas de contacto a la hora de conectarlo a la PCB y por lo tanto esta conexión tuvo que ser hecha por cables. Ambos problemas pueden verse si se observa con detenimiento el video del Ensayo final.

Para finalizar la sección de ensayos y mediciones se muestran a continuación las Figuras 6a y 6b que muestran el dashboard planteado para el sistema y una imagen del sistema en general respectivamente.

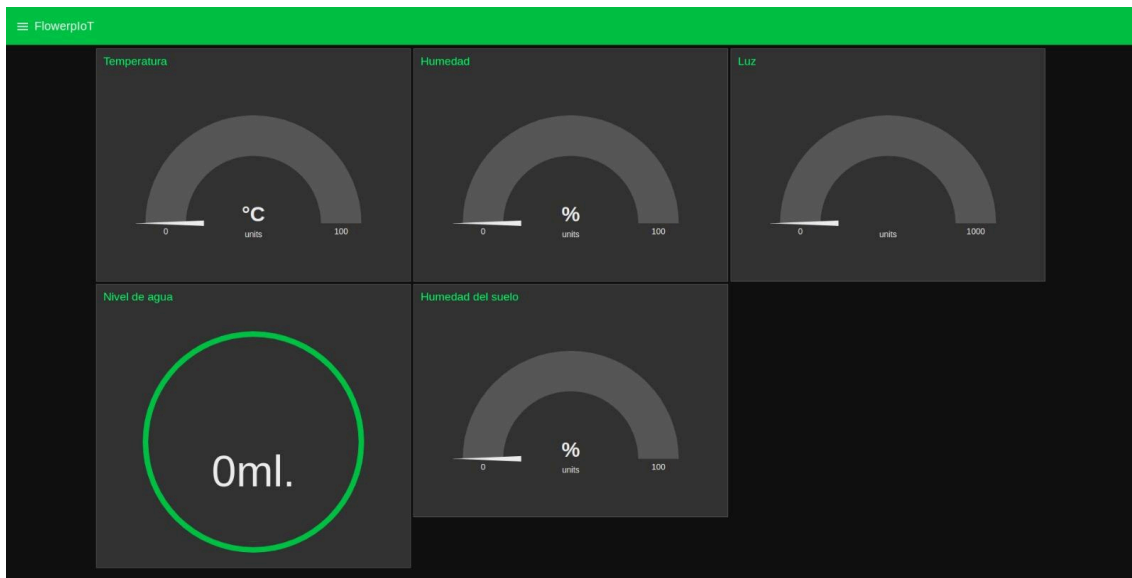


Figura 6a: Dashboard del usuario de FlowerIoT



Figura 6b: Prototipo de FlowerpIoT en funcionamiento

7. Conclusiones

El presente informe ha destacado el cumplimiento de tanto el objetivo principal, como parte de los objetivos secundarios planteados en el informe inicial.

El objetivo primario de lograr un nodo inteligente que forme parte de un sistema de monitoreo de cultivos con tecnologías de IoT para recopilar datos cruciales que afectan el crecimiento de plantas se ha cumplido. Como se ha explicado a lo largo del presente informe, esto se completó satisfactoriamente, logrando así un correcto monitoreo de los datos obtenidos desde la planta.

Como parte de los objetivos secundarios, se pudo implementar un sistema de actuación, el cual en caso de detectar baja la humedad del suelo active el sistema de riego. Además, se realizó un correcto sistema de visualización de los datos recolectados, accesible desde cualquier dispositivo. Sin embargo, algunos objetivos secundarios no fueron cumplidos, como por ejemplo el seleccionador de modo visualización/actualización, el cual había sido propuesto inicialmente y el display que se planeaba ubicar al lado de la planta. Estos objetivos secundarios no fueron cumplidos principalmente por falta de tiempo para investigación y desarrollo además de que no representaban un gran cambio respecto a lo realizado durante el trabajo. A pesar de esto, el hecho de haber cumplido no sólo con el objetivo primario, sino también con dos objetivos secundarios permite dar un cierre con un balance más que positivo.

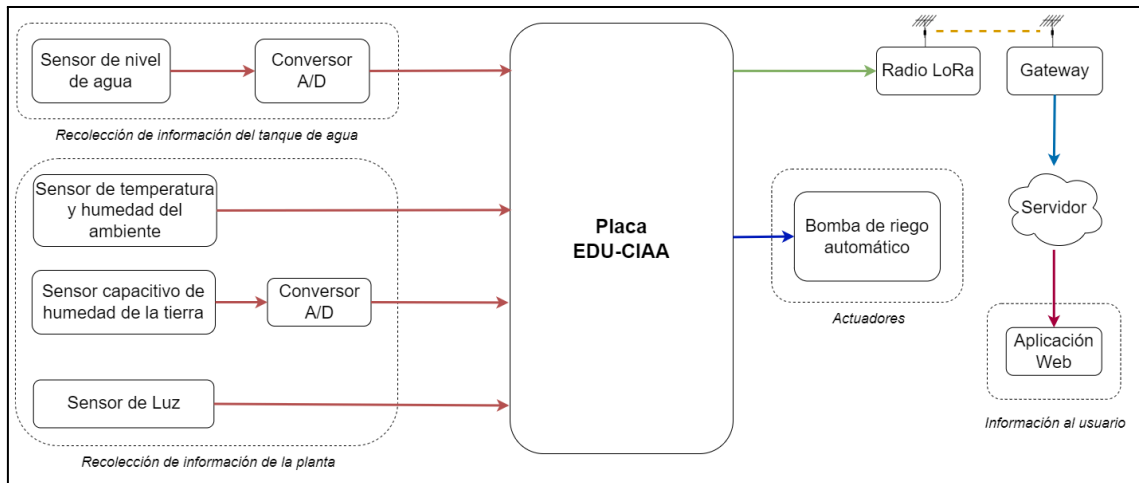


Figura 7a: Diagrama en bloques del sistema realizado

El proyecto presentó algunos cambios respecto a lo propuesto inicialmente, específicamente:

- **Ubicación de EDU-CIAA en la maceta:** Inicialmente se había propuesto hacer un compartimiento adherido a la maceta para colocar la EDU-CIAA allí. Finalmente esto se descartó dado a que se hizo uso de una maceta estándar y no una maceta impresa en 3D como se pensaba en un primer momento.
- **Consumo energético mínimo general:** Debido a que se buscaba obtener un producto mínimo viable con el cual iniciar la idea del proyecto, no se implementaron las modificaciones necesarias para que el sistema completo sea del menor consumo energético posible debido a las dificultades en el diseño que esto suponía.
- **Almacenamiento en Firebase:** Si bien ya se pensaba en el almacenamiento de una base de datos, debido a que era un objetivo primario para el correcto funcionamiento del proyecto, no se pensaba en optar por un servicio como Firebase ya que agrega más complejidad tanto en el lado del servidor como en el lado del cliente a la hora de recuperar los datos. Sin embargo, fue un extra que se acopló de manera concisa al proyecto generando que el mismo sea más escalable y robusto.
- **Alimentación:** A pesar de que en un principio se optó por alimentación a batería, con el fin de lograr un dispositivo con mayor autonomía se cambió a una alimentación a partir de una fuente

7.1 Cumplimiento de requerimientos

Con respecto a los requerimientos, tanto funcionales, no funcionales y de prueba, se analizan punto por punto para detallar su cumplimiento (o no) del mismo y un poco de su implementación. Cabe aclarar que la mayoría de estos requerimientos ya están desarrollados a lo largo del presente informe, por eso se decide no explayarse demasiado.

7.1.1 Funcionales

7.1.1.1 Recolección de datos en campo

- Recibir el valor de la humedad del suelo medido por el sensor
- Realizar el sensado del valor de la humedad del suelo de forma periódica.
- Recibir el valor de la humedad del ambiente medido por el sensor
- Realizar el sensado del valor de la humedad del ambiente de forma periódica.
- Recibir el valor de la temperatura medido por el sensor.
- Realizar el sensado del valor de la temperatura de forma periódica.
- Recibir el valor del nivel de luz medido por el sensor.
- Realizar el sensado del valor del nivel de luz de forma periódica.

Todos los requerimientos funcionales relacionados a la recolección de datos en campo fueron correctamente cumplidos. No se presentaron problemas respecto a la utilización de estos salvo por una complejidad agregada por el sensor de luz cuyo modo de comunicación no estaba incluido en la biblioteca sapi y por lo tanto hubo que desarrollarlo completamente. Esto se hizo con ayuda del Ing. Jose Juarez que proveyó el código correspondiente a la biblioteca bh1750.h.

7.1.1.2 Transmisión de Datos

- Establecer una comunicación con el gateway.
- Establecer una comunicación entre el gateway y el servidor.
- Enviar datos recopilados al servidor en tiempo real.

Se pudo realizar correctamente la comunicación, desde el envío de los datos hasta la visualización de los mismos en la aplicación. En principio se pensaba que no se iban a necesitar servicios de Network Server, debido a que se creía que el gateway podría realizar esa etapa también, lo cual no fue finalmente así y se debió hacer uso de los servicios de The Thing Stack.

7.1.1.3 Visualización de datos

- Lograr recibir a través del gateway información de niveles de luminosidad.
- Lograr recibir a través del gateway información de los niveles de humedad del suelo.
- Lograr recibir a través del gateway información de niveles de luz detectados.
- Una vez almacenados los valores, mostrar en la aplicación web, un dashboard con información actual de los sensores

Continuando con lo anterior, tras establecer una correcta comunicación entre todos los dispositivos involucrados, se logró enviar los datos recopilados para su visualización. Todos los requerimientos funcionales relacionados a la visualización de datos fueron correctamente realizados.

7.1.2 No Funcionales

7.1.2.1 Recopilación de Datos

- Garantizar una baja latencia en la recopilación y transmisión de datos.
- Minimizar el consumo de energía para prolongar la duración de la batería.

Se garantizo que la latencia de la recopilación y transmisión de datos sea acorde al proyecto. Esto quiere decir que debido a que los parámetros de temperatura, humedad, etc, en un corto periodo de tiempo no cambian, podemos no respetar una periodicidad exacta, pero si se garantizo que la latencia en recopilación y transmisión sea lo más baja posible.

En cuanto al segundo requerimiento, fue modificado por dos razones, en primer lugar, al cambiar de alimentación por batería a alimentación por una fuente la necesidad de minimizar el consumo no fue tan crucial. Por otro lado, el hecho de agregar la bomba de agua considerado originalmente como un objetivo secundario hizo que el consumo fuera mucho más alto del planteado inicialmente.

7.1.2.2 Escalabilidad

- Permitir la expansión del sistema para monitorear múltiples campos de cultivo simultáneamente.
- Garantizar que el sistema pueda manejar un crecimiento significativo en el volumen de datos.

Ambos requerimientos correspondientes a la escalabilidad fueron correctamente cumplidos, inicialmente gracias a que el gateway utilizado es multicanal, por lo cual puede recibir varios paquetes en aquellos canales, lo cual permite la escalabilidad en tanto a la recepción de datos. Por otra parte, tanto el servidor Node-RED como el Network Server (TTS) también pueden recibir múltiples paquetes de datos. En conclusión, el proyecto desarrollado es altamente escalable.

7.1.2.3 Disponibilidad

- Asegurar una alta disponibilidad del sistema para evitar interrupciones en el monitoreo.
- Establecer un plan de recuperación ante errores para mitigar fallos críticos.
- Garantizar que los sensores sean reemplazables y calibrables de forma sencilla.

El sistema no es interrumpido en ningún momento, pues se encuentra constantemente sensando y enviando los datos para su posterior procesamiento. Es decir, el sistema no se ve interrumpido en ningún momento.

Respecto a establecer un plan de recuperación: este requerimiento no fue cumplido ya que se consideró secundario debido a un cambio de paradigma establecido. Los datos no son almacenados en la EDU-CIAA, sino que se almacenan en una base de datos, que como fue explicado anteriormente, se optó por Firebase y su almacenamiento de forma no relacional, ya que no se dispone de una jerarquía importante de datos, sino que simplemente son los datos de los sensores, por eso no se requiere una estructura relacional.

Los sensores elegidos para el proyecto son simples, tanto en su conexión como en su configuración, calibración y su código de ejecución para la recolección de datos. Las bibliotecas utilizadas para el funcionamiento de los mismos ya están implementadas y sirven para diversos sensores, pues generalmente el funcionamiento de los sensores del tipo utilizado en el proyecto poseen un funcionamiento de similares características. Cabe destacar que los mismos también son reemplazables y se pueden conseguir en el mercado de manera accesible en caso de pérdida o deterioro.

7.1.2.4 Presupuesto

- Mantener el desarrollo dentro del presupuesto limitado disponible

Este requerimiento fue cumplido satisfactoriamente, debido a que se eligieron sensores adecuados para el proyecto, el cual no presentaba grandes gastos pues los sensores utilizados son de bajo presupuesto y son más que suficientes para el uso que se le buscó dar al proyecto.

7.1.2.5 Tiempo

- Alcanzar un producto finalizado en el tiempo limitado que se tiene

Gracias a la eficiencia y eficacia del grupo, se respetaron los tiempos que posteriormente serán mencionados en un diagrama de gantt donde se podrá ver al detalle la relación con respecto a la división de tareas. El producto quedó utilizable y con posibilidad de seguir desarrollándose para futuras mejoras.

7.1.2.6 Hardware

- Utilizar la placa educativa EDU-CIAA-NXP para el desarrollo del prototipo.

Este requerimiento se cumplió satisfactoriamente, ya que desde el principio el grupo se comprometió con el marco de trabajo del proyecto. Desde el primer momento, el equipo abrazó este enfoque y no surgió ningún contratiempo que impidiera su ejecución.

7.1.3 De prueba

- Documentar el proceso de calibración de sensores y configuración del sistema.
- Verificar que la comunicación Wi-Fi sea estable y no se pierda la conexión durante un periodo prolongado.
- Evaluar que el nodo inteligente pueda comunicarse con el gateway LoRa.

Se documentó correctamente con imágenes y capturas de pantalla la calibración de los sensores.

La conexión Wi-Fi es estable y el gateway no sufrió interferencias en los envíos de datos. Se realizaron pruebas de más de dos horas y los dispositivos no sufrieron fallas.

Como ya se comentó antes, se pudo establecer correctamente la comunicación desde el módulo LoRa hasta el gateway. Para las primeras comunicaciones se enviaron paquetes de prueba, incluidos en la librería provista por el fabricante del módulo LoRa, el cual nos permitió ágilmente enviar mensajes a través del protocolo, y que estos mensajes se vieran y sean recibidos y decodificados por el gateway.

- Asegurarse que el servidor en la nube pueda recibir, almacenar y gestionar los datos recibidos.
- Verificar que los datos almacenados se muestran correctamente en la plataforma web.
- Comprobar que los datos se actualicen en tiempo real.

Los datos no se actualizan en tiempo real, sino que se refrescan cada vez que el módulo LoRa envía la información. Esta frecuencia de actualización es configurable desde el código del dispositivo. Se ha determinado que un intervalo de 15 minutos es apropiado para capturar los cambios en las variables que estamos midiendo. Esto se debe a que ninguna de estas variables experimenta cambios abruptos que requieran una frecuencia de muestreo más alta. Al establecer este intervalo, podemos optimizar el uso de los recursos de comunicación y energía, ya que no hay necesidad de enviar datos con mayor frecuencia. Además, al permitir un tiempo razonable entre actualizaciones, aseguramos que los cambios significativos en las variables puedan ser detectados y registrados de manera efectiva.

7.2 División de tareas final

Siguiendo con el tema de la división de tareas del proyecto, cada integrante del equipo tuvo su rol fundamental en el transcurso del desarrollo.

- Tomás Mc Nally:
 - En primer lugar se encargó del estudio de los sensores, ensayos sobre los mismos y documentación de los resultados obtenidos. A su vez se encargó de la definición de las interfaces eléctricas y conexión de los sensores. Por otro lado fue responsable del desarrollo de los circuitos eléctricos correspondientes a la bomba de agua en conjunto con el modulo relay y la fuente.
 - Realizó un trabajo conjunto con Azul Martini en el diseño del esquemático y posteriormente de la PCB así como también del diseño 3D.
 - En total se calculan unas 79 horas de trabajo totales a lo largo del proyecto
- Azul Martini:
 - Individualmente se encargó de la definición de la topología de red a utilizar, la arquitectura del firmware y el firmware que fue desarrollado.

Al mismo tiempo se encargó del desarrollo de los algoritmos de lectura de los sensores y el diseño del código final implementando la arquitectura del firmware previamente definida.

Realizó un trabajo conjunto con Tomás Mc Nally en el diseño del esquemático y posteriormente de la PCB así como también del diseño 3D.

En total se calculan unas 78.5 horas de trabajo total en el proyecto

- Jerónimo Ratusny:
 - Realizó un trabajo conjunto junto a Mariano Rodriguez del estudio de las herramientas Node-RED y Firebase. Posteriormente, ambos se encargaron de la implementación de todos los bloques que realizan el procesamiento entre los datos enviados por el módulo LoRa, el almacenamiento de los mismos en la base de datos, y su correcta visualización en gráficos, tanto de variables actuales como históricas. Individualmente hablando, se encargó del estudio del gateway y del módulo LoRa, y de realizar su correspondiente configuración. Por último, realizó el algoritmo de comunicación entre el módulo LoRa y la EDU-CIAA.

En total se estiman unas 78 horas de trabajo total en el proyecto

- Mariano Rodriguez:
 - Realizó un trabajo conjunto junto a Jerónimo Ratusny del estudio de las herramientas Node-RED y Firebase. Posteriormente, ambos se encargaron de la implementación de todos los bloques que realizan el procesamiento entre los datos enviados por el módulo LoRa, el almacenamiento de los mismos en la base de datos, y su correcta visualización en gráficos, tanto de variables actuales como históricas. Individualmente hablando, se encargó de la implementación Gateway-Servidor, y de las pruebas de funcionamiento de comunicación desde el origen hasta el fin de la misma.

En total se estiman unas 79.5 horas de trabajo total en el proyecto

A su vez todos los integrantes del grupo fueron parte del armado del PCB y confección de los informes inicial, de avance 1, de avance 2 y final. Como conclusión se calculan unas 290 horas de ingeniería totales, sumando las 28 horas de trabajo adicionales en relación al armado del PCB.

7.3 Conclusión general

La experiencia del grupo en la materia fue muy positiva. Se logró obtener un resultado final a partir de una idea de proyecto inicial, esto pudo llevarse a cabo gracias a no sólo el trabajo del equipo, sino también a la ayuda de la cátedra que estuvo muy presente a lo largo de toda la materia, especialmente el ayudante diplomado Lucas Martire el cual fue el tutor encargado.

Él nos supo guiar en todo el camino del desarrollo debido a que la mayoría de las herramientas utilizadas no eran de conocimiento para ninguno de nosotros como lo

son la tecnología LoRa de la cual no teníamos experiencia previa, los diferentes sensores utilizados y su modo de comunicación. Entre otras cosas más relacionadas a la organización del trabajo y la división de tareas, gracias a él pudimos aprender mucho más.

El diseño y armado del PCB fue también un desafío completamente nuevo, del cual tuvimos muchas dudas al momento de diseñar y ensamblar los componentes mediante soldadura. Gracias a las charlas brindadas por la cátedra y personal del ATEI, la producción del mismo fue llevada a cabo satisfactoriamente sin mayores inconvenientes y dejando una gran experiencia.

Como grupo aportamos mucha predisposición desde el momento cero para aprender e implementar todas estas herramientas nuevas y necesarias para que el proyecto llegue a buen puerto. Nuestra organización y buen funcionamiento en equipo fue lo que nos permitió llevar a cabo gran parte de los objetivos planteados desde un comienzo para dar lugar al resultado final.

En conclusión, creemos que fue una gran experiencia enriquecedora que nos permitió adentrarnos en conceptos nuevos así como también en la forma de llevar adelante un proyecto de ingeniería.

8. Bibliografía

- [1] Pernia, Eric (2019). *Especificaciones EDU-CIAA-NXP v1.1*. Disponible en: https://github.com/epernia/firmware_v3/blob/master/documentation/CIAA_Boards/NXP_LPC4337/EDU-CIAA-NXP/EDU-CIAA-NXP%20v1.1%20Board%20-%202019-01-03%20v5r0.pdf

- [2] CIAA (2015). *Dimensiones de la placa EDU-CIAA-NXP*. Disponible en: <https://github.com/ciaa/Hardware/blob/master/PCB/EDU-NXP/Doc/PCB.pdf>

- [3] CATSENSORS (2015). *Tecnología LoRa y LoRaWAN*. Disponible en: <https://www.catsensors.com/es/lorawan/tecnologia-lora-y-lorawan>

- [4] *Sensor de Humedad y Temperatura del ambiente DHT22*. Información disponible en: <https://pdf1.alldatasheet.com/datasheet-pdf/view/1132459/ETC2/DHT22.html>

- [5] *Sensor de Humedad y Temperatura del ambiente DHT11*. Disponible en: <https://pdf1.alldatasheet.com/datasheet-pdf/view/1440068/ETC/DHT11.html>

- [6] *Sensor Capacitivo de Humedad del Suelo*. Disponible en: <https://www.digikey.com/htmldatasheets/production/2071177/0/0/1/sen0193.html>

- [7] *Sensor Analógico de Nivel de Agua*. Disponible en: https://curtocircuito.com.br/datasheet/sensor/nivel_de_agua_analogico.pdf

- [8] *MiniBomba de agua sumergible de 5v de corriente continua*. Disponible en: <https://5.imimg.com/data5/IQ/GJ/PF/SELLER-1833510/dc-mini-submersible-water-pump.pdf>

- [9] *Datasheet de Relay JQC-3F de 5V*. Disponible en: <https://pdf1.alldatasheet.com/datasheet-pdf/view/114955/ETC1/JQC-3F.html>

- [10] *Node-Red-Dashboard*. Disponible en: <https://aprendiendoarduino.wordpress.com/category/dashboard-node-red/page/3/>

- [11] *Frecuencia de LoRa en Argentina*. Disponible en: <https://www.thethingsnetwork.org/country/argentina/>
- [12] *¿Cuál es la tecnología detrás de la frecuencia LoRa?*. Disponible en: <https://www.mokosmart.com/es/lora-frequency/>
- [13] The things industries. *Tecnología LoRaWAN*. Disponible en: <https://www.thethingsindustries.com/docs/getting-started/lorawan-basics/>
- [14] *Modulación LoRa: Long Range Modulation*. Disponible en: <https://medium.com/pruebas-de-laboratorio-de-la-modulaci%C3%B3n-lora/modulaci%C3%B3n-lora-4ad74cabd59e>
- [15] *Fundamentos básicos sobre LoRaWAN*. Disponible en: <https://www.thethingsnetwork.org/docs/lorawan/>
- [16] Cátedra de Circuitos Digitales y Microcontroladores de la Universidad Nacional de La Plata (2023). *Planificación y Ejecución de Tareas en Sistemas Embebidos*.
- [17] *Repositorio github sobre el firmware de la biblioteca sAPI*. Disponible en: https://github.com/epernia/firmware_v3/tree/master

9. Anexos

Todos los anexos mencionados a lo largo del presente trabajo se encuentran dentro de la carpeta:

[Anexos](#)

Allí se tienen los anexos:

- Anexo 1: Sensores a utilizar
- Anexo 2: Información relativa al diseño del software
- Anexo 3: Ensayo sobre componentes sin PCB
- Anexo 4: Archivos relacionados al desarrollo del PCB

Además de los anexos relacionados a información relativa al desarrollo a continuación se van a desarrollar los anexos relativos al trabajo realizado por los integrantes del grupo y un manual de usuario para poder replicar las tareas mostradas si se tienen los componentes.

Por último, todo el contenido relacionado a ensayos, modelos 3D utilizados, código final, entre otras cosas puede encontrarse en:

[Informe Final](#)

9.1 División de tareas

A continuación, se especifica la división de tareas entre los diferentes integrantes del equipo:

Tabla 9.1. División de tareas realizadas por el grupo

Tarea	Responsable	Colaborador
Análisis de herramientas disponibles	Mc Nally, Tomás Ratusny, Jerónimo Martini, Azul Rodriguez Mesa, Mariano	-
Investigación protocolos LoRa y Wi-Fi	Ratusny, Jerónimo Rodriguez Mesa, Mariano	-
Elección del prototipo	Martini, Azul Rodriguez Mesa, Mariano	Mc Nally, Tomas Ratusny, Jerónimo
Busqueda de componentes	Martini, Azul Mc Nally, Tomas	Ratusny, Jerónimo Rodriguez Mesa, Mariano
Definición de Arquitectura de Firmware	Martini, Azul	-

Estudio de Topología de Red	Martini, Azul	Mc Nally, Tomás
Estudio de Sensores	Martini, Azul Mc Nally, Tomás	-
Estudio Node-RED	Ratusny, Jerónimo Rodriguez Mesa, Mariano	-
Estudio de Módulo y Gateway LoRa	Ratusny, Jerónimo Rodriguez Mesa, Mariano	Martini, Azul Mc Nally, Tomás
Definición de Interfaces Eléctricas	Ratusny, Jerónimo Mc Nally, Tomás Martini, Azul	Rodriguez Mesa, Mariano
Diseño del circuito esquemático	Mc Nally, Tomás Martini, Azul	Ratusny, Jerónimo Rodriguez Mesa, Mariano
Implementación de código de sensores	Mc Nally, Tomás	Martini, Azul
Estudio de The Things Network	Ratusny, Jerónimo Rodriguez Mesa, Mariano	-
Configuración del Gateway RAK	Ratusny, Jerónimo Rodriguez Mesa, Mariano	-
Desarrollo de código de prueba de sensores	Mc Nally, Tomás	Martini, Azul
Implementación en Node-RED	Ratusny, Jerónimo Rodriguez Mesa, Mariano	-
Correcciones al diseño esquemático	Mc Nally, Tomás Martini, Azul	-
Estudio de Firebase	Ratusny, Jerónimo Rodriguez Mesa, Mariano	
Calibración de sensores	Mc Nally, Tomás Martini, Azul	-
Implementación de Firebase	Ratusny, Jerónimo Rodriguez Mesa, Mariano	-

Realización de mediciones y ensayos	Mc Nally, Tomás Martini, Azul	-
Realización del Front-End	Ratusny, Jerónimo Rodríguez Mesa, Mariano	-
Configuración del servidor TTN	Ratusny, Jerónimo Rodríguez Mesa, Mariano	-
Diseño del circuito impreso (PCB)	Mc Nally, Tomás Martini, Azul	-
Fabricación	Mc Nally, Tomás Ratusny, Jerónimo Martini, Azul Rodríguez Mesa, Mariano	-
Ensamblaje	Mc Nally, Tomás Ratusny, Jerónimo Martini, Azul Rodríguez Mesa, Mariano	-
Pruebas de validación	Mc Nally, Tomás Martini, Azul	Ratusny, Jerónimo Rodríguez Mesa, Mariano
Informes de avance	Mc Nally, Tomás Ratusny, Jerónimo Martini, Azul Rodríguez Mesa, Mariano	-
Informe final	Mc Nally, Tomás Ratusny, Jerónimo Martini, Azul Rodríguez Mesa, Mariano	-

9.2 Cronograma final

A continuación en las Figuras 9.2a y 9.2b se muestra el cronograma de realización del proyecto junto con las horas totales invertidas por cada uno de los integrantes del equipo.

SEMANA	21-ago	28-ago	4-sept	11-sept	18-sept	25-sept	2-oct	9-oct	16-oct	23-oct
	1	2	3	4	5	6	7	8	9	10
PROYECTO: ETAPA DE INVESTIGACION										
Análisis de herramientas disponibles										
Investigación protocolos LoRa y Wi-Fi										
Elección del prototipo										
Busqueda de componentes										
PROYECTO: ETAPA DE DISEÑO										
Definición de Arquitectura de Firmware										
Estudio de Topología de Red										
Estudio de Sensores										
Estudio Node-RED										
Estudio de Módulo y Gateway LoRa										
Definición de Interfaces Eléctricas										
Diseño del circuito esquemático										
Implementación de código de sensores										
Estudio de The Things Network										
Configuración del Gateway RAK										
Desarrollo de código de prueba de sensores										
Implementación en Node-RED										
Correcciones al diseño esquemático										
Estudio de Firebase										
Calibración de sensores										
Implementación de Firebase										
Realización de mediciones y ensayos										
Realización del Front-End										
Configuración del servidor TTN										
Diseño del circuito impreso (PCB)										
PROYECTO: ETAPA FINAL										
Fabricación										
Ensamblaje										
Pruebas de validación										
Muestra grupal										
ENTREGAS FORMALES										
Realización informe inicial			X							
Informe Avance 1							X			
Informe Avance 2										
Informe final										
Horas sem de clase	4	2	4	2	2	4	2	4	2	4
Total horas de clase	4	6	10	12	14	18	20	24	26	30

30-oct	6-nov	13-nov	20-nov	27-nov	4-dic	11-dic	5-feb	12-feb	Horas Invertidas			
11	12	13	14	15	16	17	18	19	Azul Martini	Tomas Mc Nally	Mariano Rodriguez	Jerónimo Ratusny
									4	4	4	4
									0	0	6	6
									6	3	6	3
									4	4	2	2
									8	0	0	0
									4	1	0	0
									6,5	12	2	2
									0	0	10	10
									2	1	9,5	9
									6	10	4	7
									5	9	6,5	4
									4	5	1	1,5
									0	0	3	2
									0	0	2	1
									2	4	0	0
									0	0	6	5
									3	2	0	0
									0	0	2,5	1,5
									4	3	0	0
									0	0	2	4
									2	2	0	0
									0	0	2	2
									0	0	4	4
									8	7	0	0
									3,5	4	2	3
									2,5	2	2	4
									2	4	1	1
									2	2	2	2
									78,5	79	79,5	78
	X					X						
4	2	4	2	2	4	4	4	2				
34	36	40	42	44	48	52	56	58				

9.3 Manual de Usuario

En la siguiente sección se detallarán los pasos a seguir para poder utilizar el nodo IoT con el objetivo pensado, incluyendo desde la conexión del nodo a la instalación y visualización de los datos una vez obtenidos.

El manual fue realizado bajo el supuesto de que se recibe la EDU-CIAA-NXP con el PCB directamente conectado junto con los sensores y la fuente de alimentación. Si se tiene el código ya cargado dirigirse al apartado 9.3.2 Conexionado y alimentación, en caso contrario continuar el orden propuesto para poder cargar el código deseado.

9.3.1 Carga de código en la EDU-CIAA-NXP

En primer lugar dirigirse al IDE Eclipse⁴, una vez allí crear la carpeta del proyecto (a modo de ejemplo para el caso propuesto es tdp1) y dentro de esta incluir las carpetas que se encuentran en la carpeta:

▢ Código Final

Dentro de esta carpeta se encuentra el archivo main el cual puede ser editado según el código deseado por el usuario. Una vez hecho esto crear y/o editar en caso de que ya se encuentre creado los archivos board.mk y program.mk respectivamente para que contengan las siguientes líneas:

- board.mk:
BOARD = edu_ciaa_nxp
- program.mk:
PROGRAM_PATH = examples/c
Nombre de la carpeta del proyecto, en nuestro caso tdp1
PROGRAM_NAME = tdp1

Una vez hecho esto, debugear el código presionando en el botón “Debug Configurations” presente en la sección “Run” previamente habiéndolo configurado según lo que se puede observar en las Figuras 9.3.1a,b y c.

⁴ Si se siguieron las notas de instalación propuestas debería estar dentro de C:\CIAA\CIAA_Software_1.1-Win\CIAA-Launcher.exe

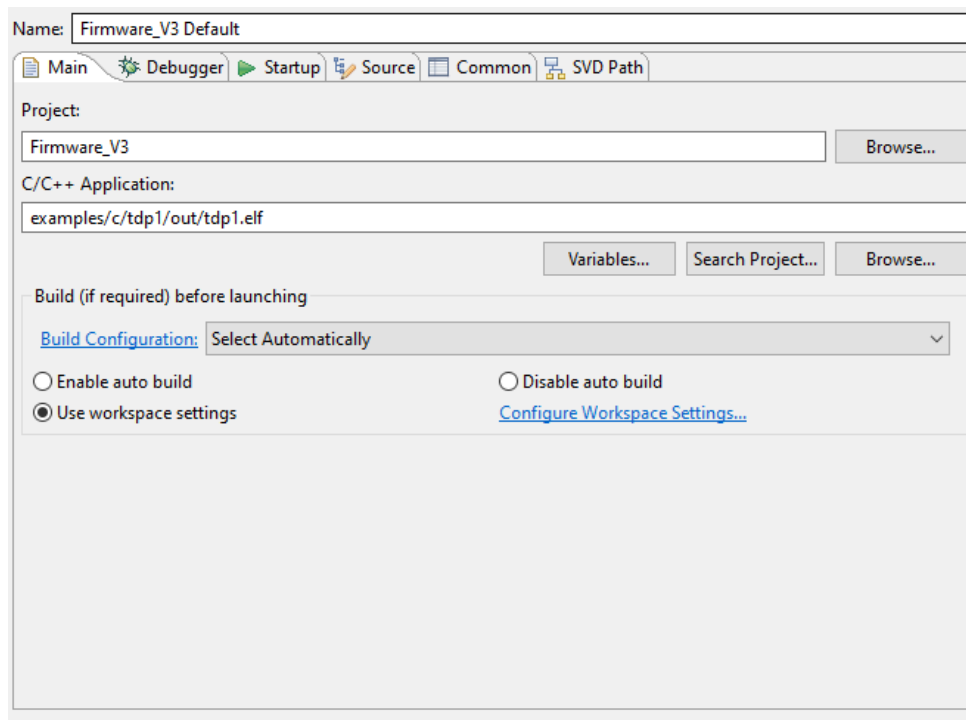


Figura 9.3.1a. Configuración correspondiente a la pestaña Main dentro de Debug Configurations

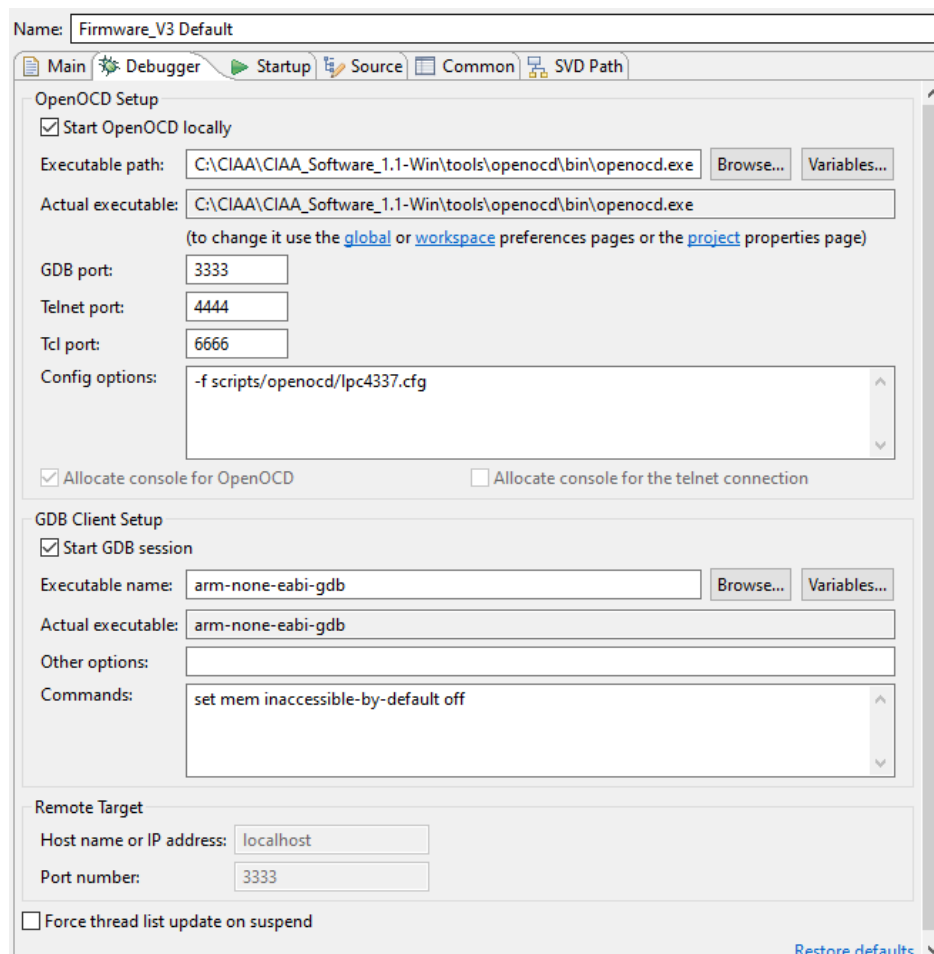


Figura 9.3.1b. Configuración correspondiente a la pestaña Debugger dentro de Debug Configurations

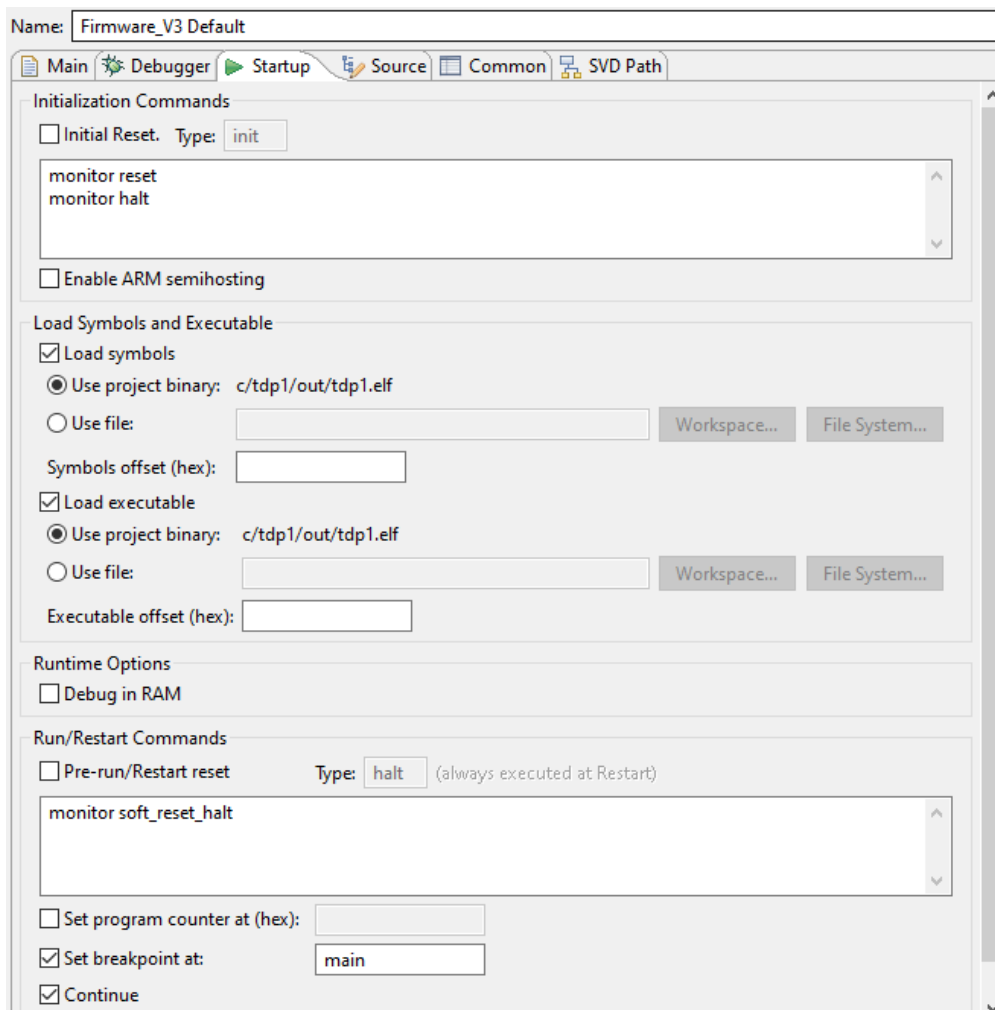


Figura 9.3.1c. Configuración correspondiente a la pestaña Startup dentro de Debug Configurations

Por último presionar el boton Debug para debuggear el código realizado. Una vez obtenido un resultado satisfactorio, conectar a través de un cable microusb la EDU-CIAA-NXP a un puerto USB de la computadora y presionar el boton Build para cargar el código a la placa de desarrollo. Una vez completados estos pasos ya se tiene el código cargado en la placa de desarrollo, para finalizar desconectar la placa de la computadora y dejarla conectada a la fuente y los componentes a través del PCB.

9.3.2 Conexionado y alimentación

Ya teniendo una placa de desarrollo con el código deseado y los componentes conectados directamente lo único que se debe realizar es conectar la fuente a la corriente y ya funciona completamente el nodo IoT, en las siguientes secciones se abordarán los pasos a seguir para poder configurar el gateway y el dashboard para poder observar los datos enviados.

9.3.3 Configuración y alimentación del Gateway

Primero se debe conectar el gateway tanto a corriente como a internet. Como bien aclaramos en el Anexo 2, apartado 4, existen varias formas de alimentar el dispositivo y también de proveer internet. Por otra parte, deberá ingresar a la configuración del dispositivo, para ello, primero desde un celular o computadora, conectar a su red Wi-Fi, que tiene como nombre clave “RAK” y posteriormente un número que representa el nombre completo de ID. Por último, al igual que a la hora de configurar un módem en el hogar, ingresará en un navegador la dirección: “192.168.230.1” y esa dirección lo redirigirá a la página de configuración del gateway.

Una vez en la configuración debemos seguir el Anexo 2, apartado 4.1.2.1 en donde explica en qué modo debe ir el gateway. Para finalizar, cabe destacar que posteriormente explicaremos qué dirección debemos de poner en el campo “Server Address”, ya que esa dirección y los puertos, nos la provee TTS.

9.3.4 Configuración de TTS

Deberá crear una cuenta en TTS, una vez creada y configurada, creará un proyecto con el nombre que desee. Se le desplegará una vista así:

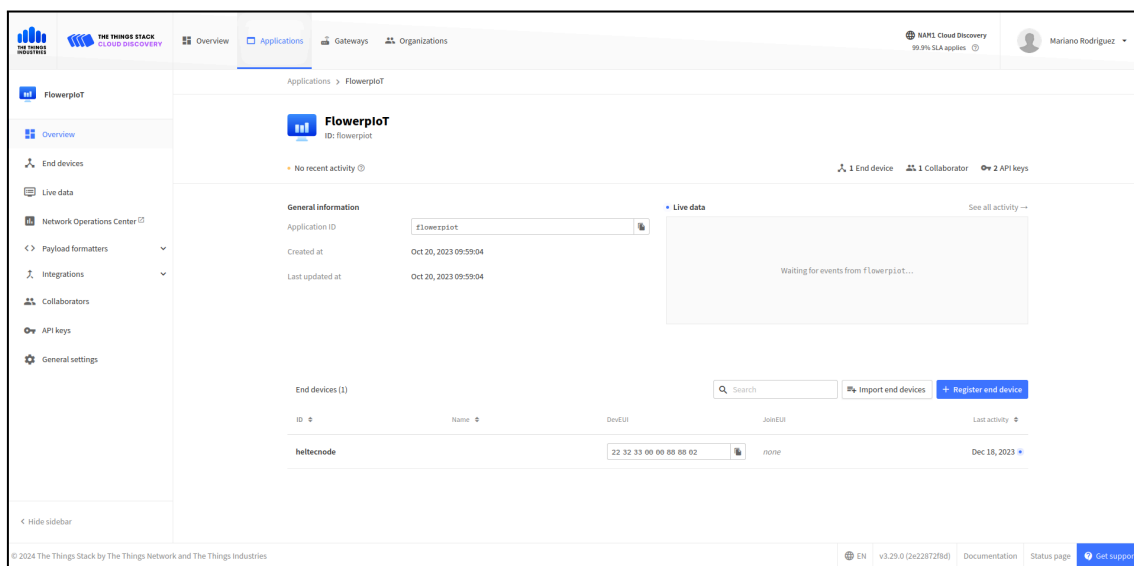


Figura 9.3.4a. Dashboard de aplicación creada en TTS

Luego, podrá ver abajo a la derecha un botón azul que dice “Register End Device”, ahí registrará su nodo LoRa con algunos datos que le pedirá como plan de frecuencia, DevEui, entre otros datos.

Posterior a eso, debemos crear una conexión MQTT con Node-RED, en la figura xx, podrá ver a la izquierda la opción de integraciones, que adentro tiene la opción MQTT, allí tendrá la dirección pública para conectarse con TTS, y además, deberá poner un usuario y generar una API Key para poder acceder, ya que sino cualquier usuario con la dirección pública podría entrar a su aplicación y podría vulnerar la misma.

MQTT

MQTT is a publish/subscribe messaging protocol designed for IoT. Every application on TTS automatically exposes an MQTT endpoint. In order to connect to the MQTT server you need to create a new API key, which will function as connection password. You can also use an existing API key, as long as it has the necessary rights granted.

Further resources

[MQTT server](#) | [Official MQTT website](#)

Connection information

MQTT server host

Public address

Public TLS address

Connection credentials

Username

Password

Figura 9.3.4b. Configuración protocolo MQTT en TTS

Esas credenciales posteriormente se le indicará donde ingresarlas en el servidor Node-Red. Cabe destacar que también tiene más al detalle cómo se realizó para el proyecto en el Anexo 2, apartado 6.1.

9.3.5 Configuración del Servidor Node-RED

Primero deberá instalar node-red, que puede seguir las instrucciones de la siguiente dirección: [Running Node-RED locally](#). Una vez instalado deberá correr el comando “node-red” para ejecutar el servidor. La dirección del mismo se le informará en la consola, pero de forma predeterminada la dirección es “127.0.0.1:1880”. Una vez en la pantalla de configuración de Node-Red, deberá importar el archivo llamado “flowerpIoT.json” para tener todos los componentes que hacen funcionar la aplicación. Las bibliotecas utilizadas dentro de Node-Red son:

- node-red-contrib-firebase: provee nodos para la conexión con Firebase.
- node-red-contrib-ui-media: provee nodos para el dashboard.
- node-red-dashboard: se encarga de crear el dashboard en la dirección “127.0.0.1:1880/ui”

Esas son las bibliotecas que se deben instalar previamente para poder tener todos los nodos del archivo y que no hayan errores. Luego, deberá en el nodo MQTT, configurar según lo que indique TTS

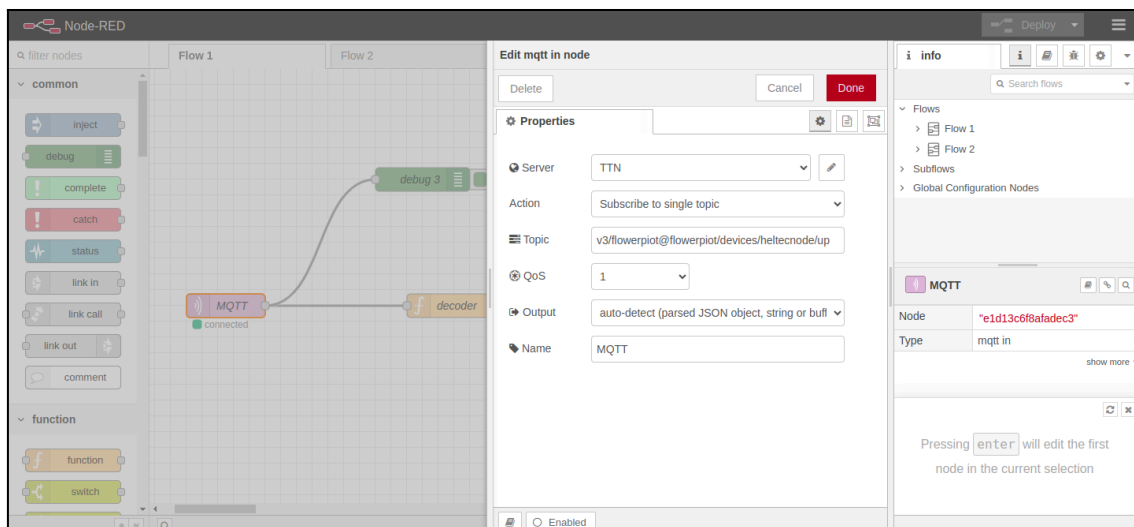


Figura 9.3.5a. Configuración protocolo MQTT en Node-Red

En donde dice “Server” deberá poner los datos y credenciales que le indica TTS en la figura xx.

Por último, para conectar Firebase, en el nodo correspondiente, es muy parecido a MQTT, en la configuración del nodo de firebase, se debe especificar a que Firebase se le quiere hacer consultas. En el siguiente apartado, se le indicará dónde puede conseguir ese link. Cuenta con más información acerca de Node-Red en el Anexo 2, apartado 6.2.

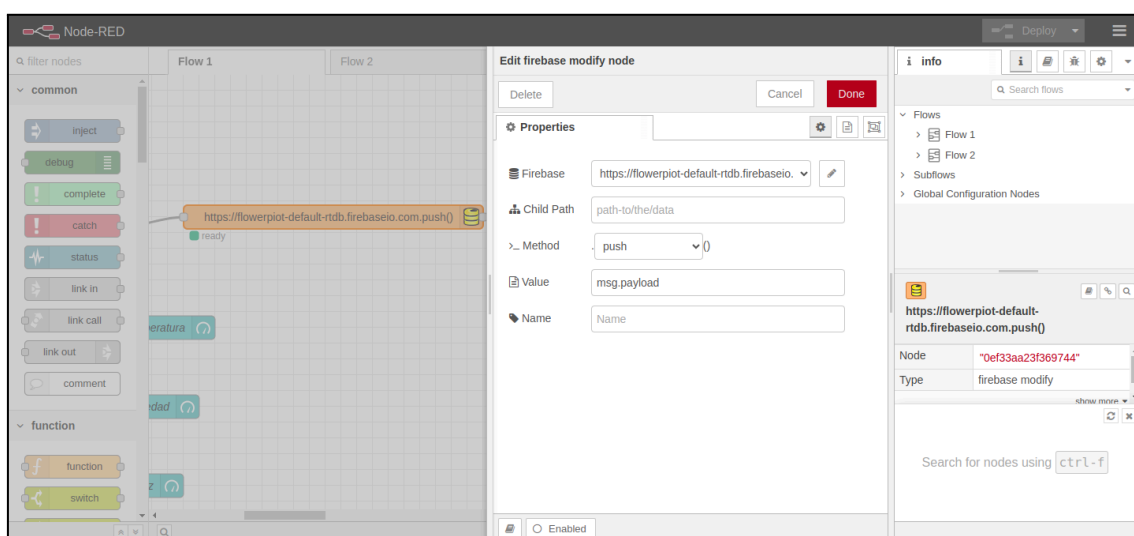


Figura 9.3.5b. Configuración Firebase en Node-Red

9.3.6 Configuración de la Base de Datos Firebase

Firebase es un servicio de google, por lo que sí dispone de una podrá acceder de forma gratuita y fácil al servicio en cuestión. Una vez que se haya registrado o iniciado sesión, tendrá una pantalla donde están sus proyectos. Allí podrá crear sus proyectos, luego deberá darle de alta a una “Realtime Database”, donde encontrará el link de

conexión. que deberá poner donde se le indico en el apartado de “Configuración del servidor Node-Red”.

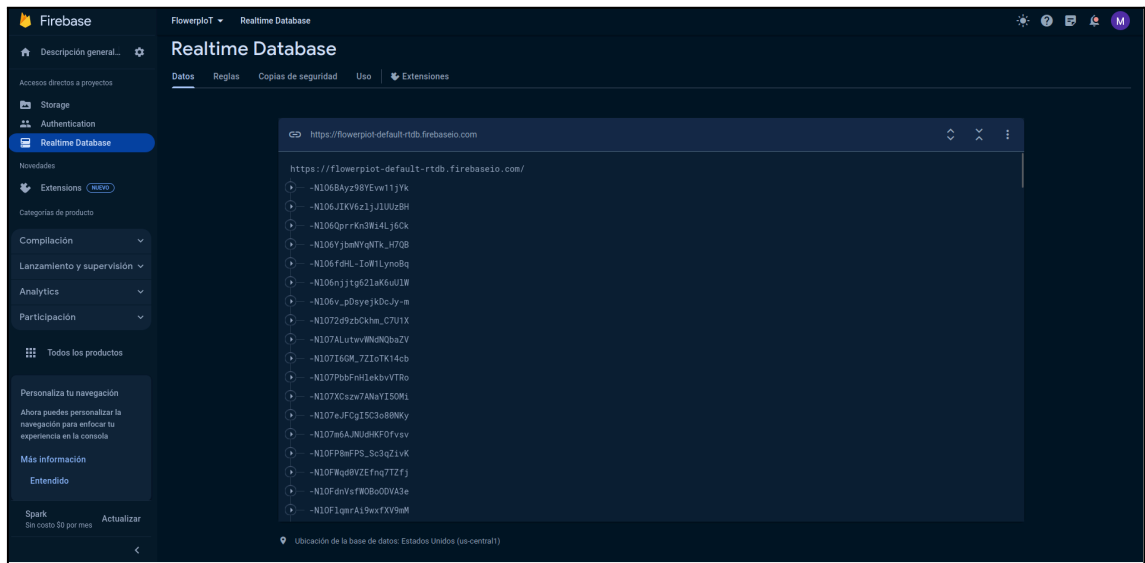


Figura 9.3.6a. Configuración Firebase en Node-Red

Para finalizar, una vez seguidos todos estos pasos se obtiene un nodo completamente funcional listo para usar.